

先進計算機構成論 04

東京大学大学院 情報理工学系研究科 創造情報学専攻

塩谷 亮太

shioya@ci.i.u-tokyo.ac.jp

前回の振り返り：工場のラインを考える



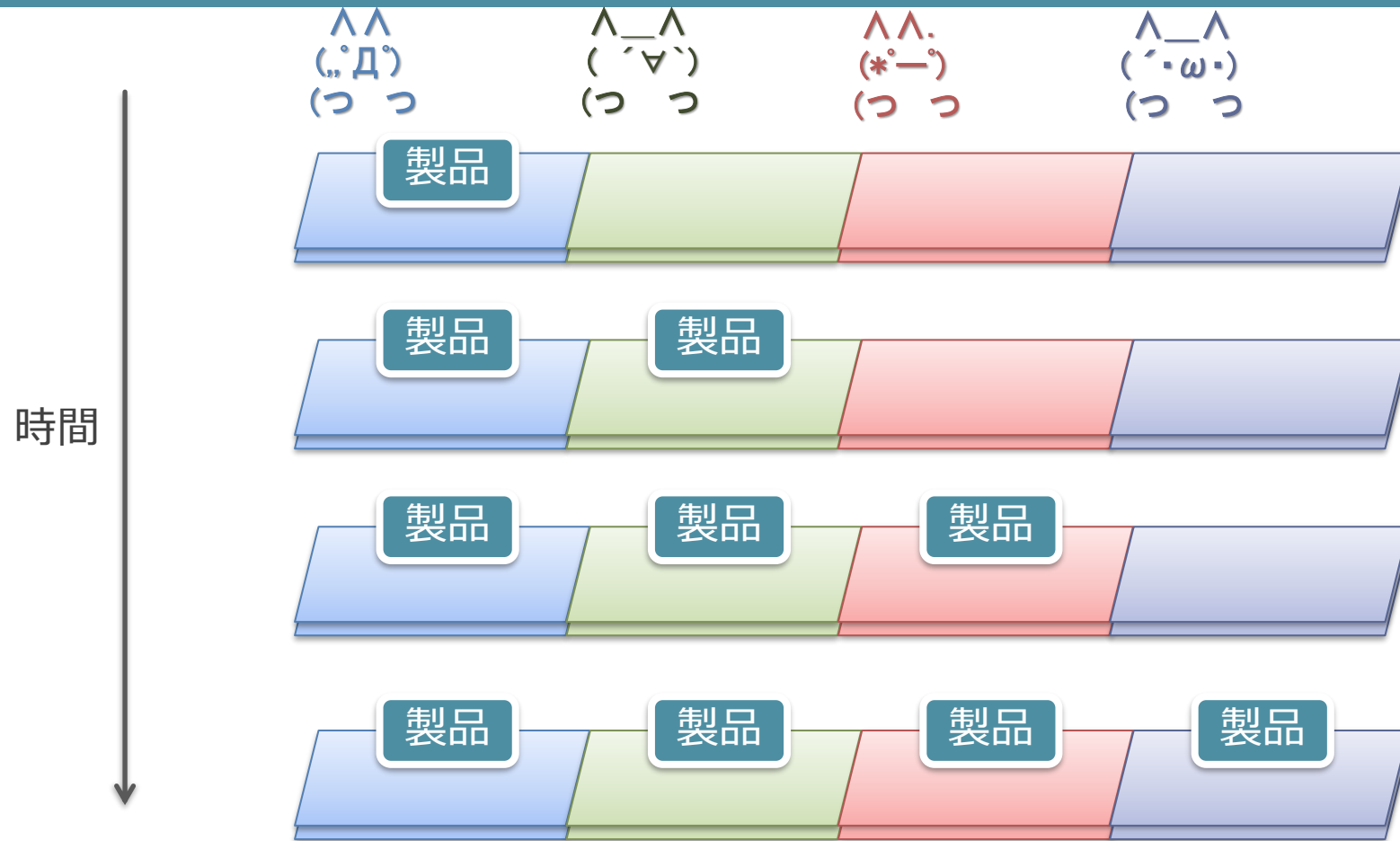
- ベルトコンベアのラインの上を製品が流れていく
 - 4 人の人が、それぞれの工程の作業をおこなって完成
- 上のように1つしか製品をながさないで、
 - 各人は他の人が作業している間はヒマ

前回の振り返り：工場のラインを考える



- 複数の製品を同時に流す
 - 各工程を並列して処理することによりスループットを向上
 - 4倍の速度で製品ができあがっていく

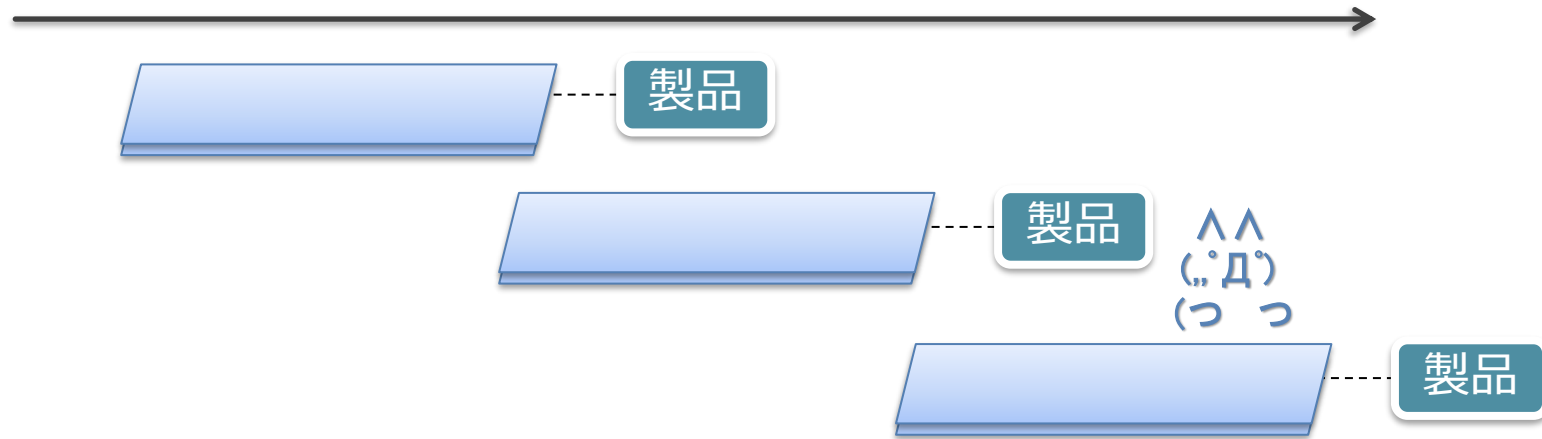
前回の振り返り：工場のラインを考える



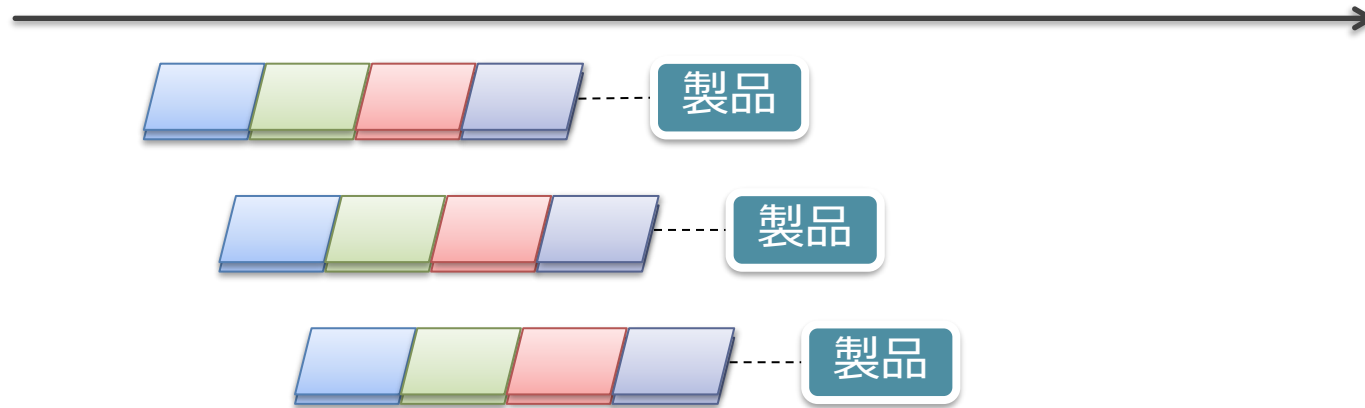
- 命令パイプライン も同じ考え方に基づく

パイプライン化による性能向上

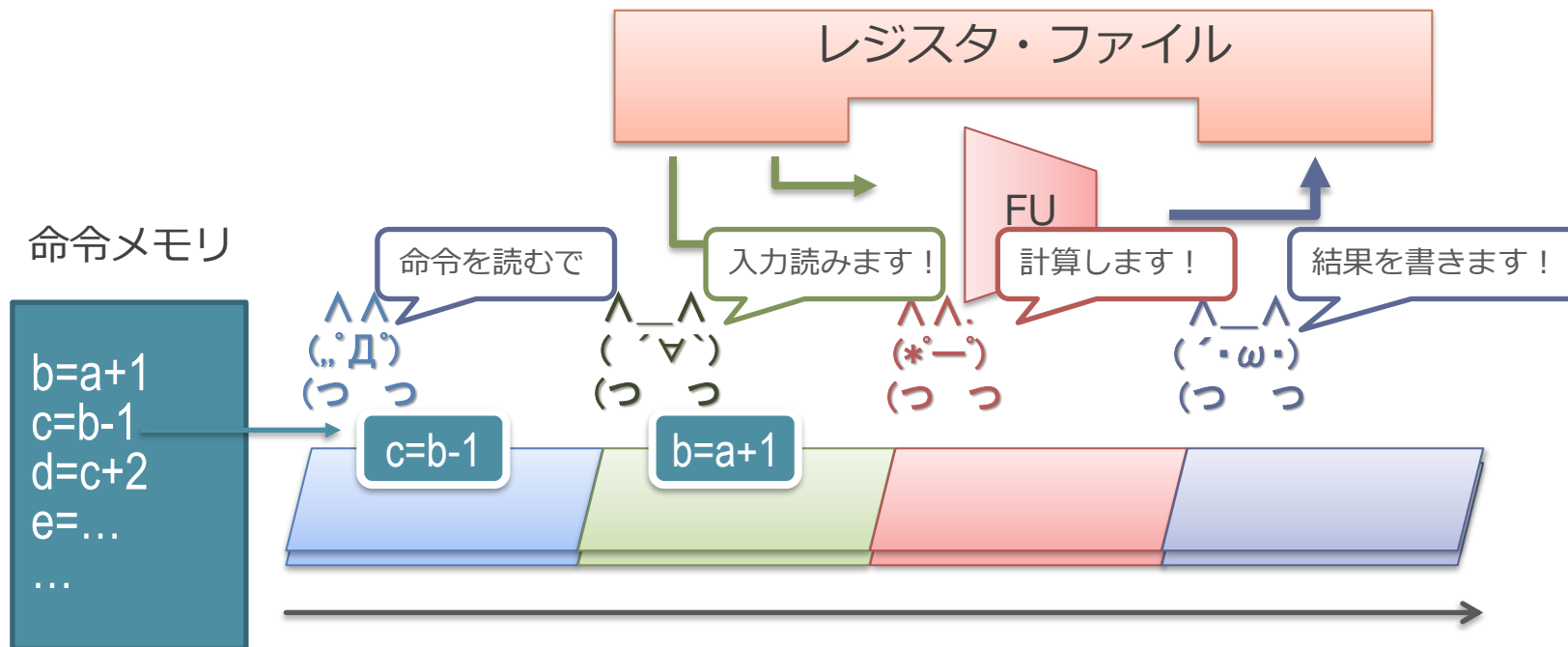
パイプライン化しない場合



パイプライン化した場合



命令パイプライン



- (, D) が命令をメモリから読み出す
- (' ∇) がレジスタ・ファイルから入力の値を読み出す
- (* -) が演算器 (FU) で計算する
- (' ω) がレジスタ・ファイルに結果を書き込む

ハザード

ハザード (hazard)

- パイプラインがうまく動作しないこと
 - パタヘネ（教科書）の定義：
「パイプラインにおいて次のサイクルに次の命令を実行できないこと」
- ハザードの種類：
 - 構造ハザード
 - 非構造ハザード
 - データ・ハザード
 - 制御ハザード

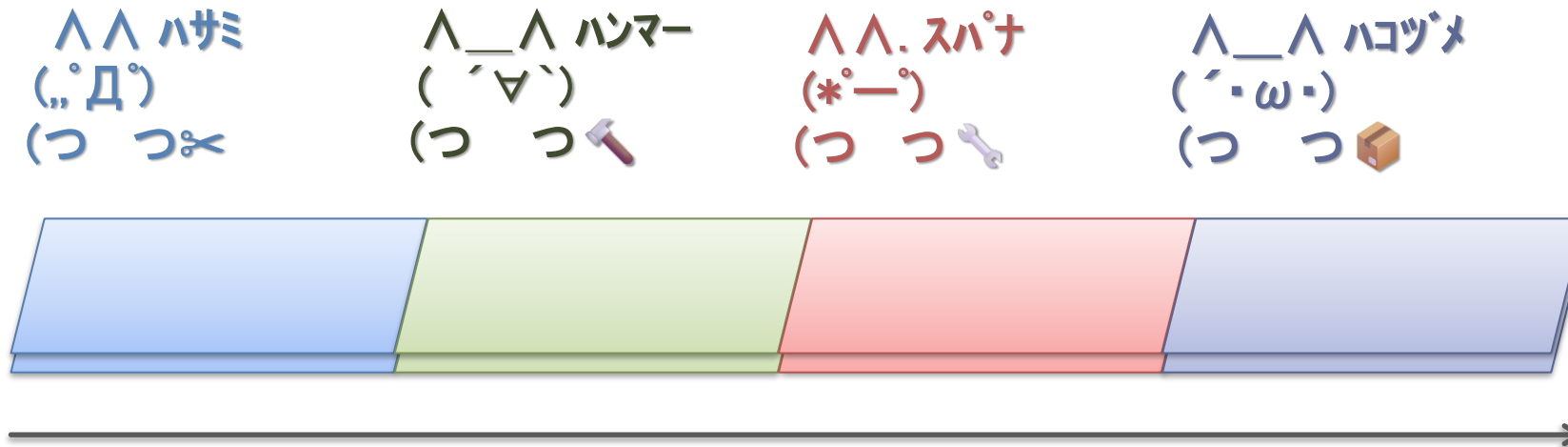
1. 構造ハザード：ハード資源の不足に起因

1. 構造ハザードとはなにか？
2. その解決方法

2. 非構造ハザード：バックエッジに由来

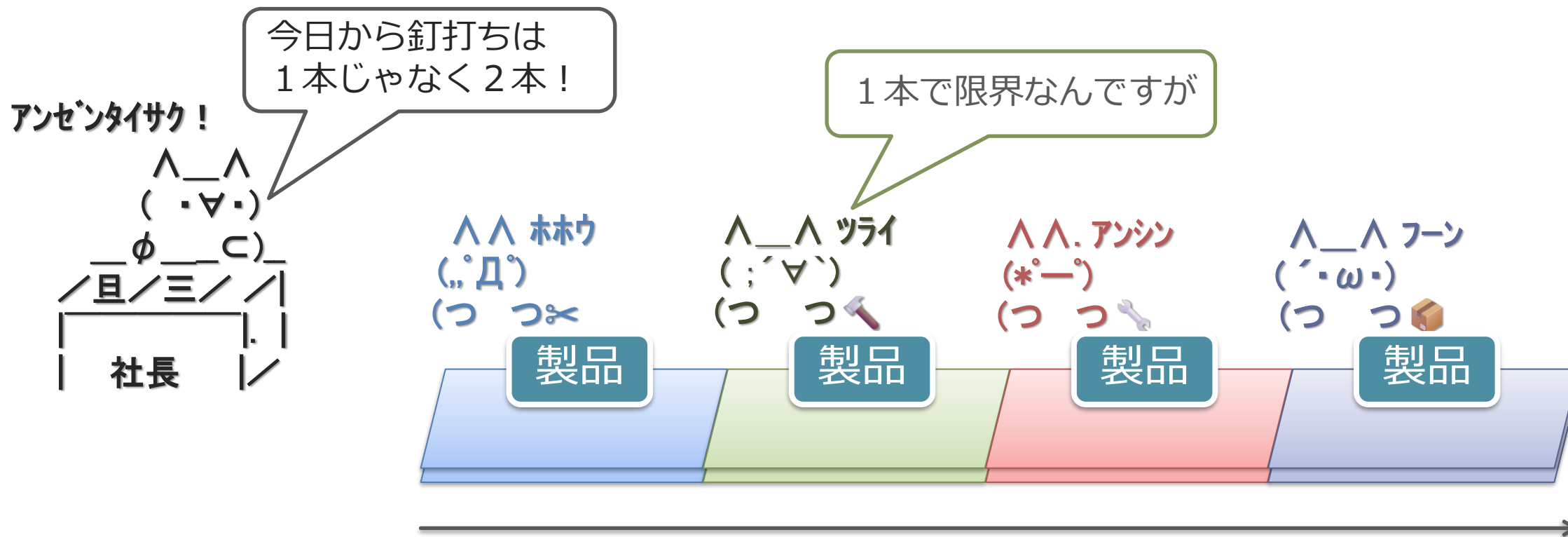
- a. データ・ハザード
- b. 制御ハザード

工場のラインをまた考える



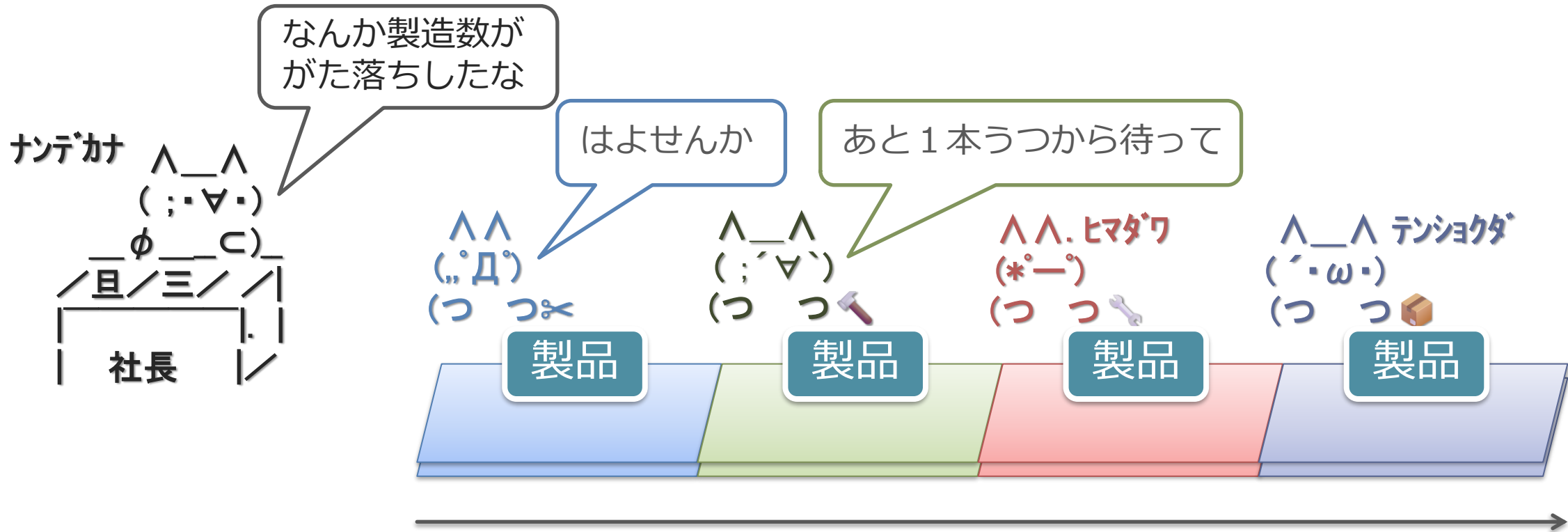
- 各ステージの担当者は製品が流れるまでの間に一定の仕事ができる
 - はさみで紙を1枚切る
 - ハンマーで釘を1本打つ
 - スパナでねじを1個しめる
 - 箱に製品を1つ入れる

釘の本数の変更



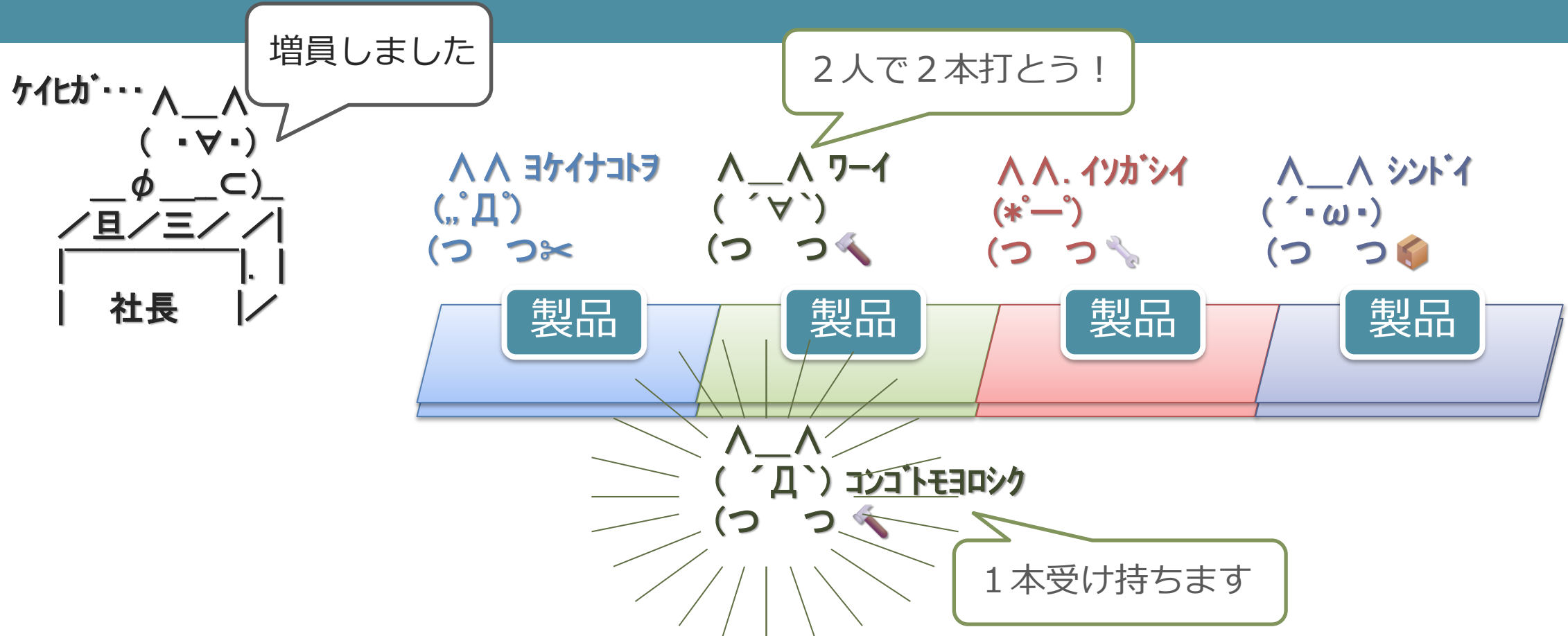
- 製品の安全対策のために釘の本数が1本から2本に
 - しかし、ハンマーを持っている釘打ち担当は単位時間に1本しか打てない

釘打ちの人のところで構造ハザード



- 2人目(;・▽・)の人は単位時間に1本しか打てない
 - ラインが一番遅い人に合わせて動く（前回の講義より）
 - そこで2本打つまでライン全体が止まる
 - 全体の速度がそこで決まってしまう

増員による構造ハザードの解消



- 1人増やして緑のステージで単位時間に2本の釘が打てるように
 - これがハード資源の追加による構造ハザードの解消
 - ただし、追加しただけ経費がかかる
 - (ハードだとその分複雑になって電力を食う)

コンピュータにおける構造ハザード

■ ハード資源：

- 演算器（FU）, レジスタ・ファイル, メモリ など

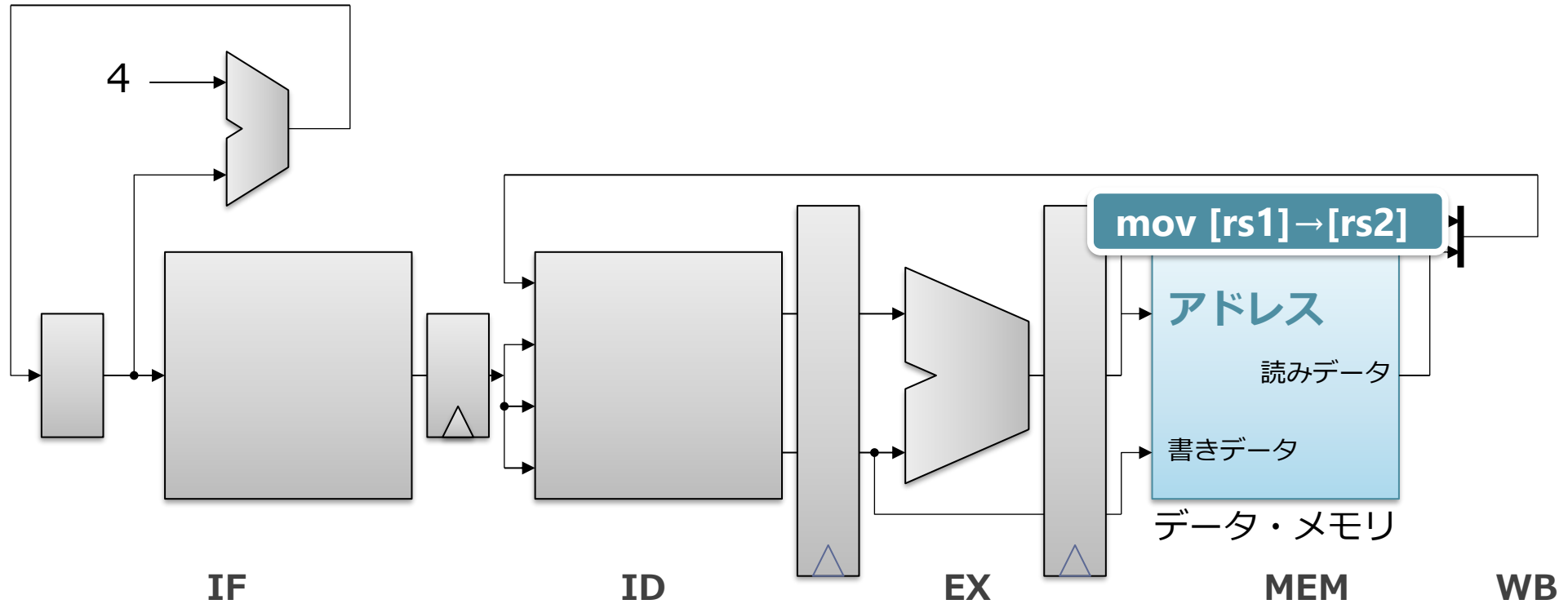
■ 構造ハザード：

- ハード資源の不足により, パイプラインがうまく動作しないこと
- いくつかの例を使った説明, 解消方法について解説

構造ハザードの例 1 : メモリ間 mov

- 例 1 : 仮に `mov [rs1]→[rs2]` のような命令があったとする
 - `rs1` で指定されるアドレスのメモリの値を読んで,
 - `rs2` で指定されるアドレスのメモリに書き込む
- 実際に, x86 にはこのような命令がある

mov [rs1]→[rs2] // [rs1]→[rs2] へのコピー



- メモリをあるサイクルに同時に読んで書く必要がある
 - しかし, データ・メモリのアドレスの口は1つしかない
 - MEM ステージでデータ・メモリの読みと書きが同時にできない

構造ハザードの例 2 : push/pop

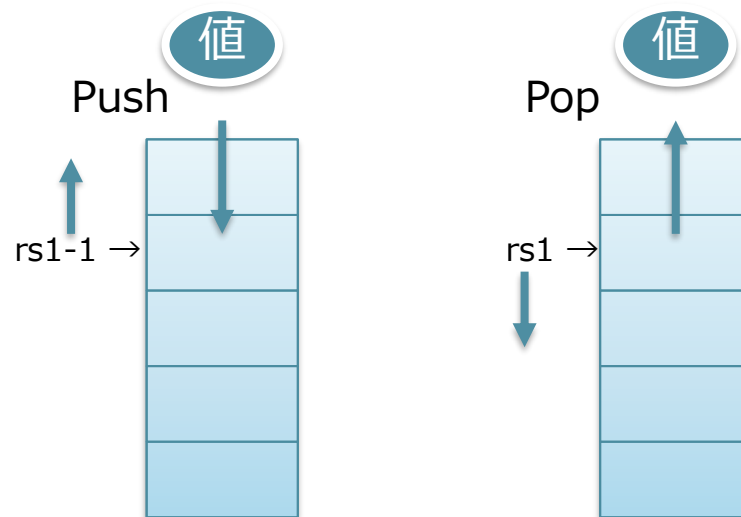
■ x86 や ARM ではスタック操作のための push/pop 命令がある

- push : $rs1-1 \rightarrow rd$, $r2 \rightarrow [rd]$

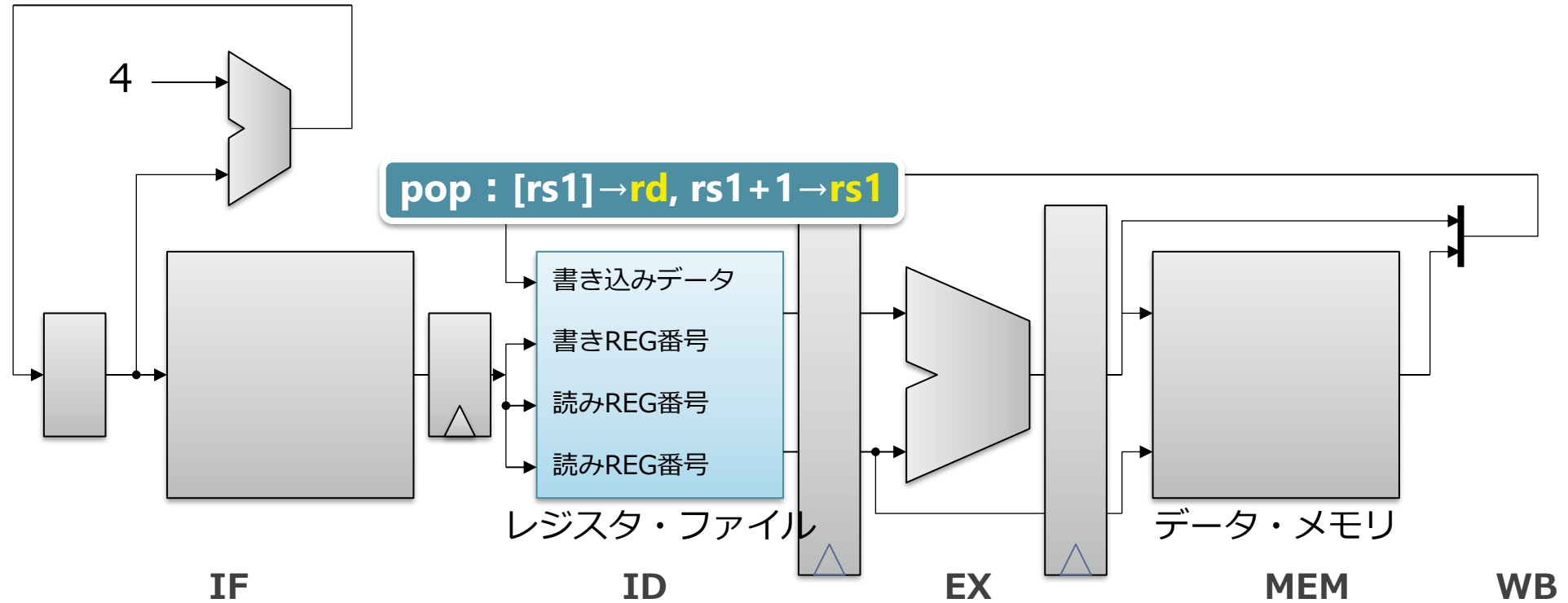
1. スタック・ポインタ（のレジスタ）をデクリメントし,
2. それをアドレスにしてメモリに値を書き込む

- pop : $[rs1] \rightarrow rd$, $rs1+1 \rightarrow rs1$

1. スタック・ポインタをアドレスにして値を読む
2. スタック・ポインタをインクリメント



pop : [rs1]→rd, rs1+1→rs1



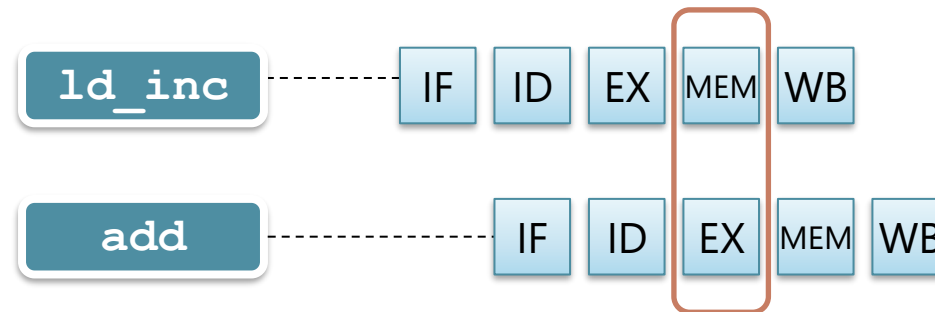
- WB ステージでレジスタに rd と rs1 の2つを書き込む必要がある
 - レジスタ・ファイルへの書き込みは、同時に2つはできない

構造ハザードの例 3

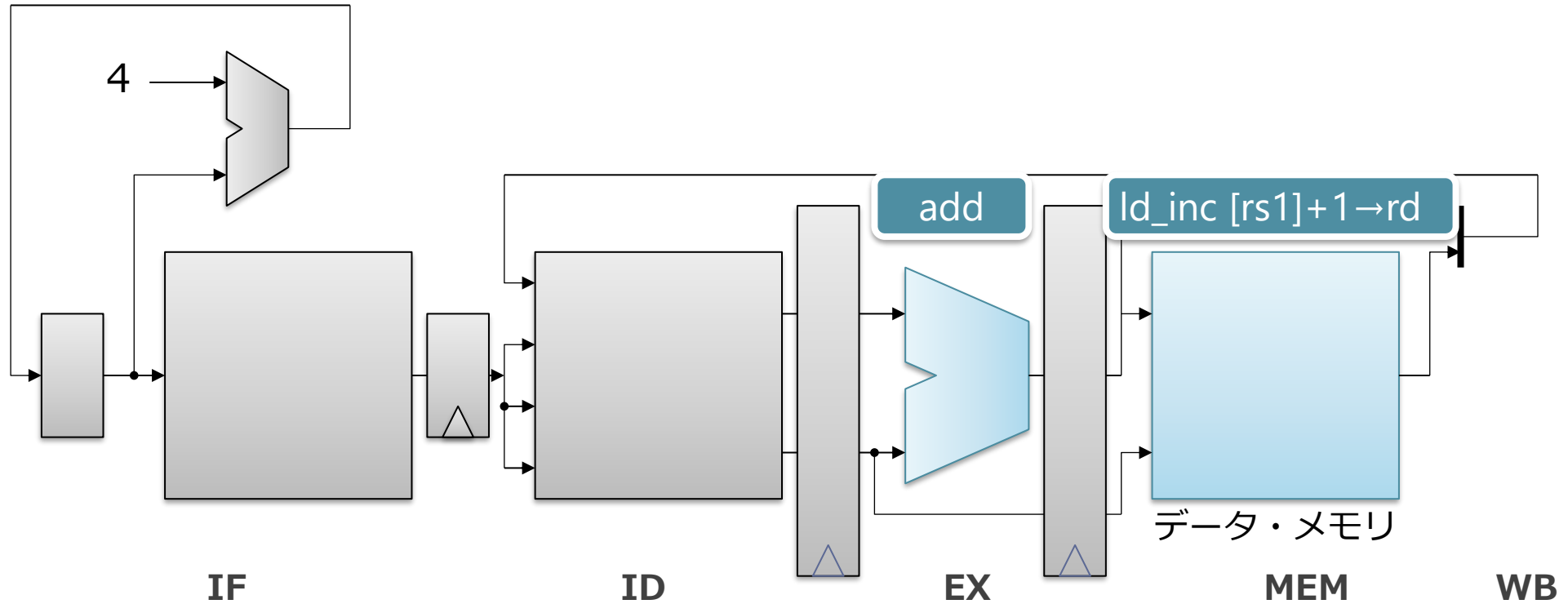
- 使用資源の異なるステージ間のぶつかりでも起きる
 - これまでの例は, 同じステージ内で資源が足りない例
- `ld_inc [rs1]+1→rd` のような命令があったとする
 1. `rs1` の指すアドレスからメモリを読む
 2. 読んだ値にさらに + 1 してから `rd` に書く
- 一見, 資源は足りているようだが…
 - レジスタの読み書きは 1 つずつしかない
 - メモリも 1 カ所を読むだけ
 - 加算も 1 回行っただけ

構造ハザードの例 3

- `ld_inc [rs1]+1→rd` と `add` が連続した場合：
 - `ld_inc` で, MEM ステージから読んだ値を加算しようとしても,
 - そのサイクルは後続の `add` が演算器を使っているので使用できない



ld_inc [rs1]+1→rd と add が連続した場合



- EX ステージ以外では、演算器にはアクセスできない
 - 他の命令が使っている可能性がある

構造ハザードの解決方法

■ 解決方法

1. ハードウェアの増強
2. 時分割処理
3. マイクロ命令への変換

解決方法 1 : ハードウェアの増強

■ ハードウェアを増強する

- `mov [rs1]→[rs2]`
 - 複数箇所のメモリを同時に読み書きできるように
- `pop`
 - レジスタに 2 つ同時に書き込めるように
- `ld_inc [rs1]+1→rd`
 - MEM ステージに専用の加算器を追加

解決方法 1 : ハードウェアの増強

- 利点 : オーバーヘッドをいとわなければ, 基本これで解決
- 欠点 : 回路規模が増える
 - 1. 機能の増強量に比例した回路が必要
 - なにも考えないで対応していくと, ものすごい数の回路になる
 - 例 : ARM は全 16 レジスタを一気にメモリに書ける命令がある
 - 2. 機能の増強量に対して, 線形より大きなオーダーで回路規模が増える場合もある
 - 加算器などなら, 増やした数の分だけ線形に回路が増える
 - メモリやレジスタは, 同時に読み書きできる数の 2 乗で回路が大きくなる (今後の講義で説明)

ARM（32ビット）の Load/Store Multiple (LDM/STM) 命令

ビットマスクで指定した最大16個のレジスタへのロードストアを行う

関数呼び出し/復帰の際の、レジスタの保存や復帰でよく使われる

A3.12 Load and Store Multiple instructions

Load Multiple instructions load a subset, or possibly all, of the general-purpose registers from memory.

Store Multiple instructions store a subset, or possibly all, of the general-purpose registers to memory.

Load and Store Multiple instructions have a single instruction format:

LDM{<cond>}<addressing_mode> Rn{!}, <registers>{^}
STM{<cond>}<addressing_mode> Rn{!}, <registers>{^}

where:

<addressing_mode> = IA | IB | DA | DB | FD | FA | ED | EA

31	28	27	26	25	24	23	22	21	20	19	16	15	0
cond		1	0	0	P	U	S	W	L	Rn		register list	

register list	The list of <registers> has one bit for each general-purpose register. Bit 0 is for R0, and bit 15 is for R15 (the PC).
	The register syntax list is an opening bracket, followed by a comma-separated list of registers, followed by a closing bracket. A sequence of consecutive registers can be specified by separating the first and last registers in the range with a minus sign.
P, U, and W bits	These distinguish between the different types of addressing mode (see <i>Addressing Mode 4 - Load and Store Multiple</i> on page A5-41).
S bit	For LDMs that load the PC, the S bit indicates that the CPSR is loaded from the SPSR after all the registers have been loaded. For all STMs, and LDMs that do not load the PC, it indicates that when the processor is in a privileged mode, the User mode banked registers are transferred and not the registers of the current mode.
L bit	This distinguishes between a Load (L==1) and a Store (L==0) instruction.
Rn	This specifies the base register used by the addressing mode.

A3.12.1 Examples

```
STMFD R13!, {R0 - R12, LR}
LDMFD R13!, {R0 - R12, PC}
LDMIA R0, {R5 - R8}
STMDA R1!, {R2, R5, R7 - R9, R11}
```

構造ハザードの解決方法

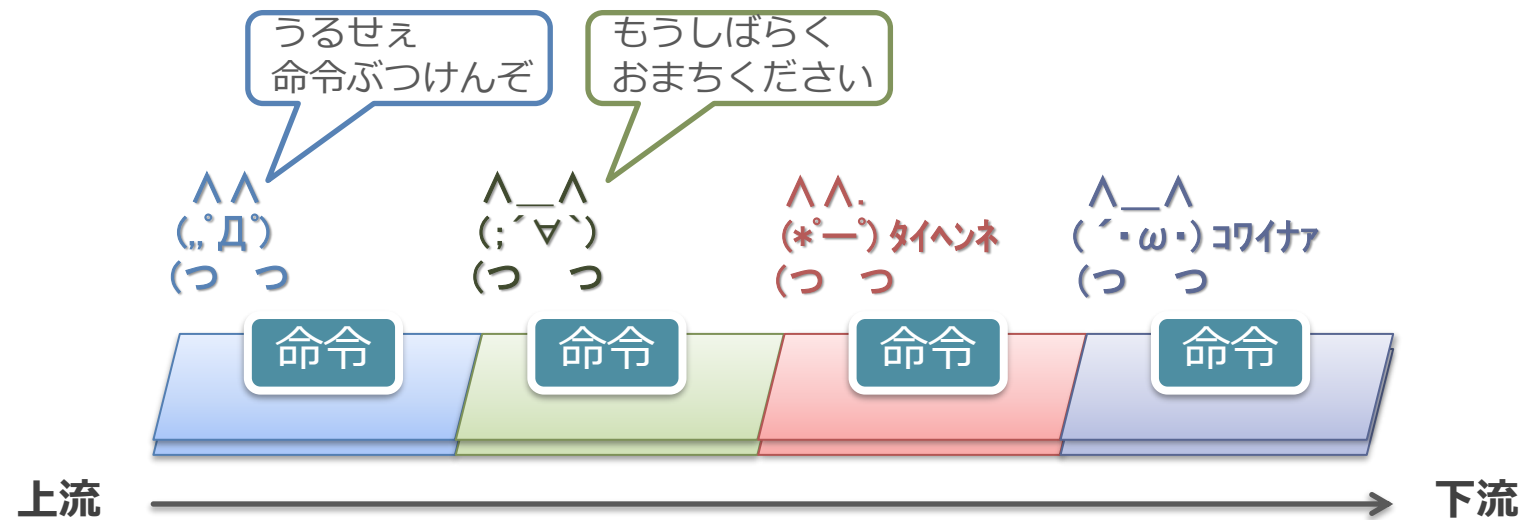
■ 解決方法

1. ハードウェアの増強
- 2. 時分割処理**
3. マイクロ命令への変換

解決方法 2 : 時分割で処理

- 構造ハザードの原因：
 - ハードウェア（の機能）が足りない
- パイプラインを止めて、複数のサイクルをかけて処理する
 - `mov [rs1]→[rs2]`
 - メモリを読んだあと、次のサイクルで書きこむ
 - `pop`
 - 1つレジスタに書いたあと、次のサイクルで書き込む
 - `ld_inc [rs1]+1→rd`
 - `ld_inc` が MEM で値を読んだら、次のサイクルで +1

なぜパイプラインを止めるのか

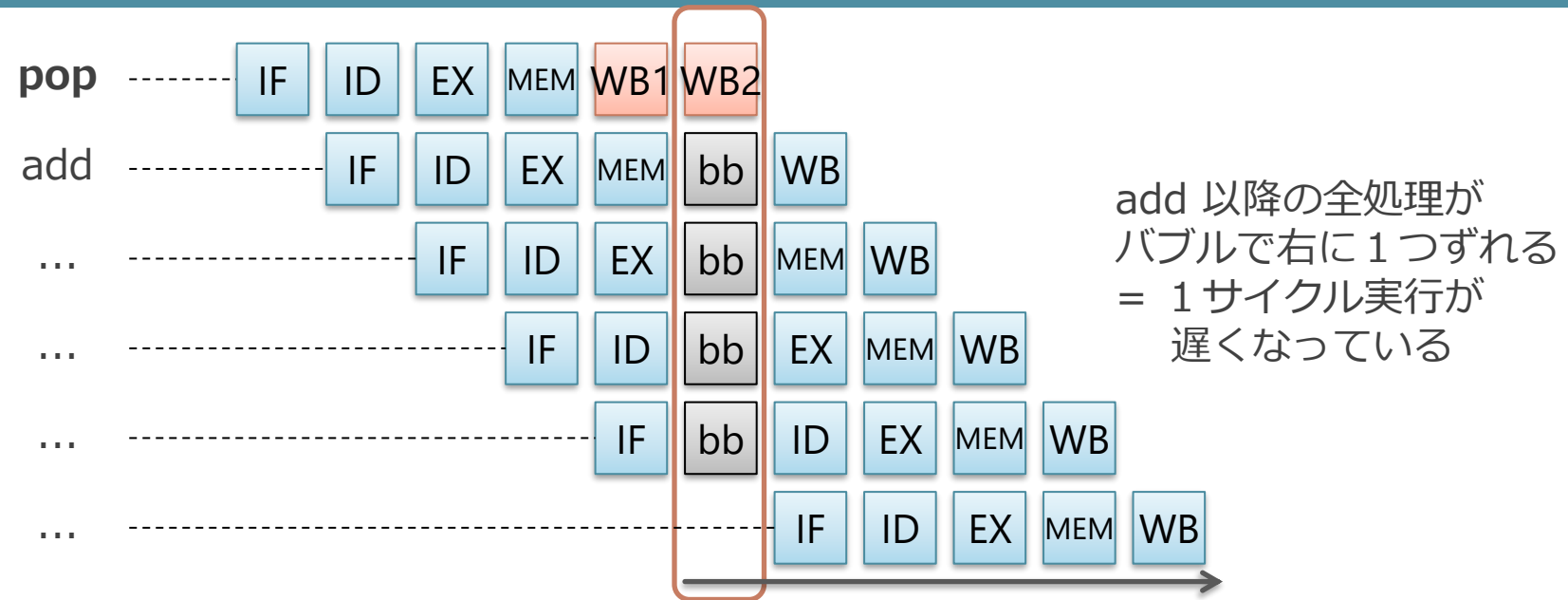


- 上流を止めないと破綻する
 - $(;\prime\vee\backslash)$ が複数サイクルをかけて仕事をしている場合、命令はそこにとどまり続ける
 - その間は上流をとめないと命令をおく場所がないし、依存関係がまもられない
- $(*\prime-)$ より下流は流れていっても、この場合は問題ない

パイプラインを止めること

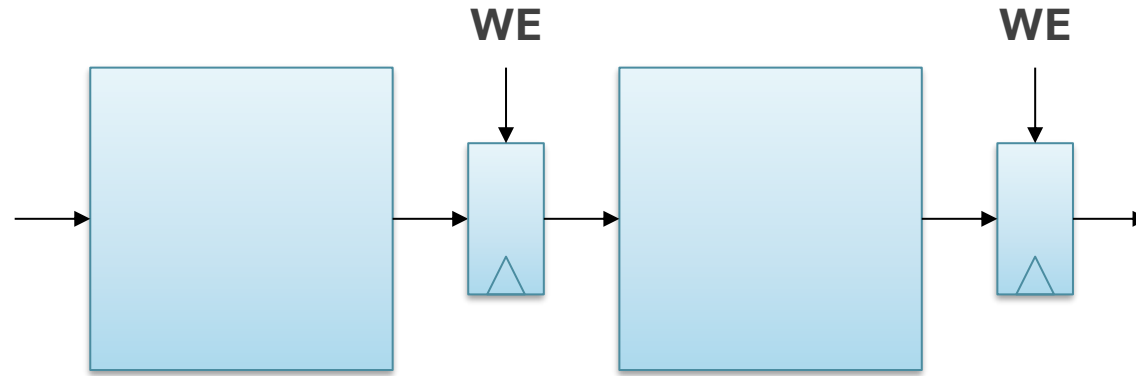
- パイプラインを止めるとことを「ストール」や「インターロック」という
 - 本や人によって、意味や使い方が微妙に統一されていない
 - この講義では、以降はストールで統一

ストールの動作



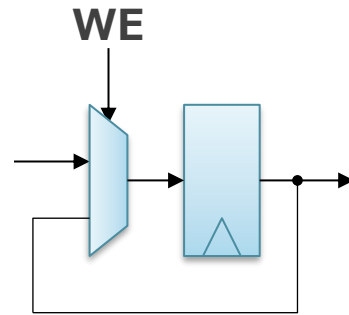
- pop : 1つレジスタに書いたあと、次のサイクルで書き込む
 - WB1 と WB2 の2サイクルで書き込む
 - WB2の間は上流を全て止める
- パイプライン・チャート上では上記のようになる
 - 止める原因の命令の下が全部右にずれる
 - ずれた部分の空き (bb) を「バブル」とよぶ

ストールの実現方法



- 回路的には、Write Enable (WE) 付きの D-FF を使う
 - WE が 0 のサイクルは書き込みが行われない
 - ストールさせたい時は、そのステージの WE を 0 に

WE つき D-FF の実現方法



- たとえば D-FF とマルチプレクサで作れる
 - WE が 0 の時は, その時の自分自身の出力を書き込む

構造ハザードの解決方法

■ 解決方法

1. ハードウェアの増強
2. 時分割処理
3. **マイクロ命令への変換**

解決方法 3 : マイクロ命令への変換

- 複数のマイクロ命令に分解して実行
 - マイクロ命令 : CPU の内部でのみ使われる命令
 - プログラマからは全く見えない
 - マイクロ命令は, 構造ハザードを起こさないよう設計しておく
- 現代の x86 や ARM は, 主にこの方法を採用している

マイクロ命令への変換の例

■ mov [rs1]→[rs2]

1. ld [rs1]→rt
2. st rt→[rs2]

■ pop

1. add rs1+1→rt
2. ld [rt]→rd

■ ld_inc [rs1]+1→rd

1. ld [rs1]→rt
2. add rt+1→rd

rt はプログラマから見えない CPU 内部にある中間結果を保持するレジスタ

マイクロ命令の分解の例

- 解析結果をまとめているサイトも

<https://uops.info/table.html>

下の例は、x86-64 の CMOVBE 命令の、Alder Lake 内の高性能コアにおける分解の説明

CMOVBE (R64, M64)

Summary: "Conditional Move"
Reference: <https://www.felixcloutier.com/x86/CMOVcc.html>
Extension: BASE
Category: CMOV
ISA-Set: CMOV
CPL: 3
iform: CMOVBE_GPRv_MEMv
iclass: CMOVBE
ASM: CMOVBE

Operands

- Operand 1 (w): Register (RAX, RCX, RDX, RBX, RSP, RBP, RSI, RDI, R8, R9, R10, R11, R12, R13, R14, R15)
- Operand 2 (r): Memory
- Operand 3 (r, suppressed): Flags (CF: r, ZF: r)

Available performance data

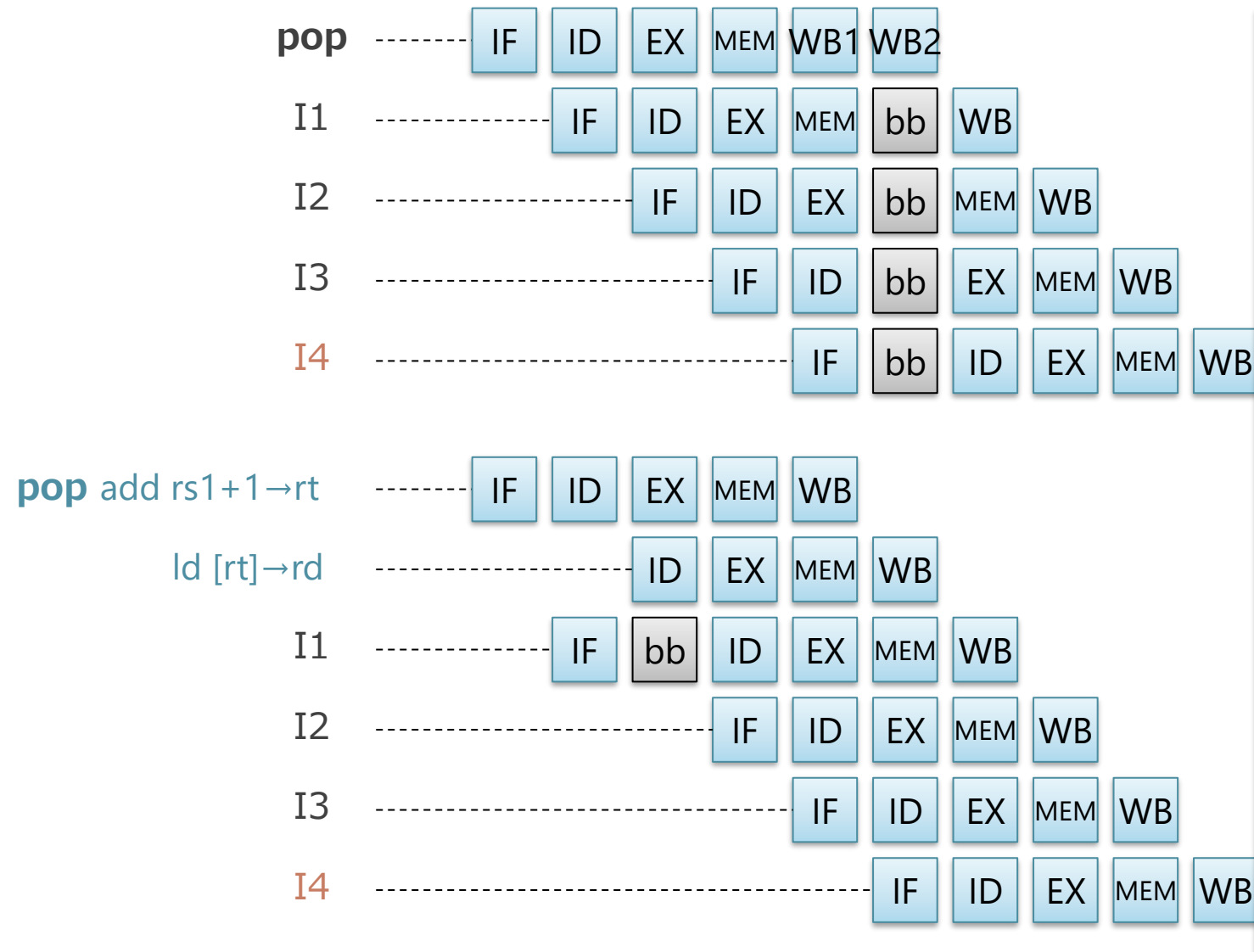
- [Alder Lake-P](#)
- [Alder Lake-E](#)
- [Rocket Lake](#)
- [Tiger Lake](#)

Alder Lake-P

- Measurements
 - Latencies
 - [Latency operand 1 → 1](#): 1
 - [Latency operand 2 → 1 \(address, base register\)](#): 6
 - [Latency operand 2 → 1 \(address, index register\)](#): 6
 - [Latency operand 2 → 1 \(memory\)](#): ≤6
 - [Latency operand 3 → 1](#): 2
 - Throughput
 - Computed from the port usage: 1.00
 - [Measured \(loop\)](#): 1.02 (if an indexed addressing mode is used: 1.00)
 - [Measured \(unrolled\)](#): 1.00
 - [Number of uops](#)
 - Executed: 3
 - Retire slots: 2 (if an indexed addressing mode is used: 3)
 - Decoded (MITE): 2
 - Microcode Sequencer (MS): 0
 - Requires the complex decoder (4 other instructions can be decoded with simple decoders in the same cycle)
 - [Port usage](#): 2*p06+1*p23A

時分割処理とマイクロ命令への分解の比較

I4 が終わる時間は変わらない



- ID でマイクロ命令に分解 = デコードで時分割処理している

マイクロ命令への分解の利点

- 処理時間が変わらないのなら、なぜこんな複雑なことをするのか？
- 分解後は、構造ハザードのことを一切考えなくてよくなるから
 - `mov`, `pop`, `ld_inc` が連続で来た場合、どう止めたらよいのか？
 - 止めるべきステージの場所はさまざま
 - 組み合わせると意味がわからない
 - マイクロ命令に分解してしまえば、ID ステージでのストールのみ考えれば良い

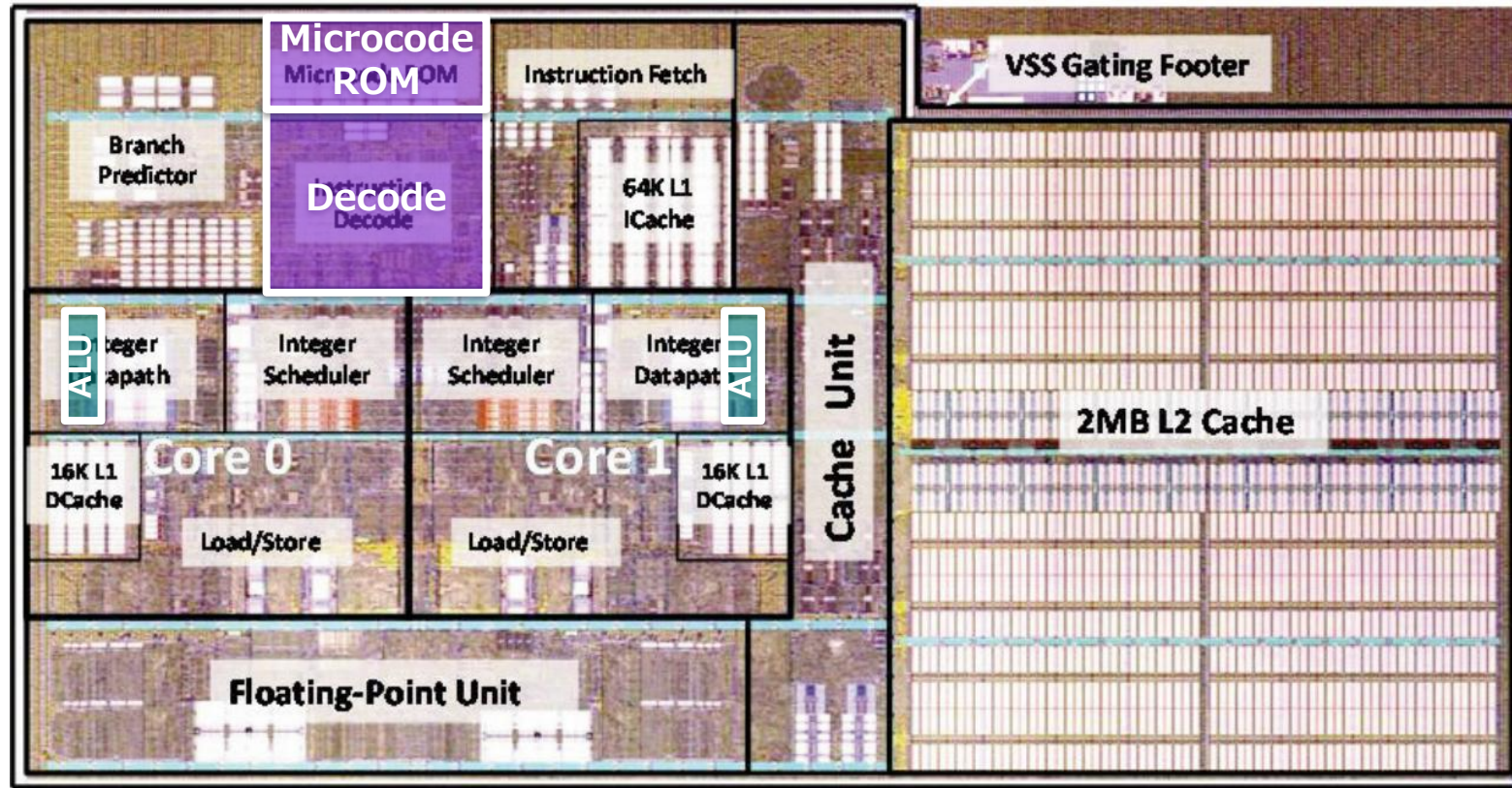
マイクロ命令への分解の利点

- 内部の設計をクリーンにできる
 - スーパスカラ（パイプラインを複数並列に並べる）などでは、
こうしないと複雑すぎて無理
- = 内部を刷新しつつ、プログラムの互換性を保てる

マイクロ命令への分解の欠点

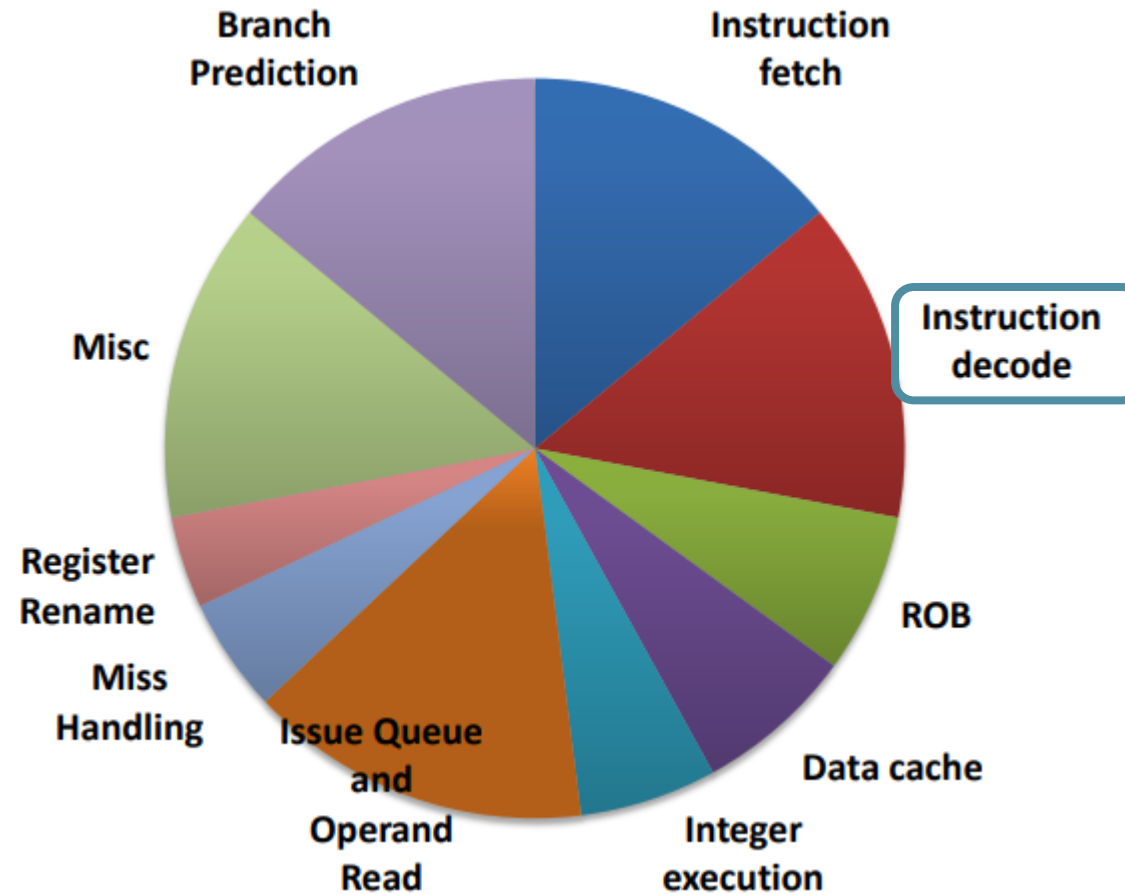
- 分解（デコード）がマジで大変
- 基本的には、ひたすらパターン・マッチング
 - でかい真理値表がいる
 - 本当に複雑なものは、メモリで出来たテーブルも使う

AMD Bulldozer のチップ写真



- Tim Fischer¹, Srikanth Arekapudi², Eric Busta¹, Carl Dietz³, Michael Golden², Scott Hilker², Aaron Horiuchi¹, Kevin A. Hurd¹, Dave Johnson¹, Hugh McIntyre², Samuel Naffziger¹, James Vinh², Jonathan White⁴, Kathryn Wilcox, Design Solutions for the Bulldozer 32nm SOI 2-Core Processor Module in an 8-Core CPU, ISSCC 2011 より

ARM Cortex-A15 の消費電力の割合



■ NVIDIA Tegra 4 Family CPU Architecture より

マイクロ命令への分解の他の利点

- CPU にバグがあったときに，後からパッチが当てられる
 - バグを「エラッタ」とも呼ぶ
- 動作がおかしい命令を，違う命令の列で置き換える
 - 分解に使う表は，あとから書き換えられるようになっている
 - Microcode ROM というのがそれ
 - ただし，元の命令の動作を他の複数の命令で再現したりするので，かなり遅くなる事もある

インテルの Core シリーズのエラッタのリスト

地味に結構バグってる

Errata Summary Table

Erratum ID	Processor Line/Stepping							Title
	UP3	IOT UP3	UP4	H35	H81	UP3-Refresh	H35-Refresh	
TGL001	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	X87 FDP Value May be Saved Incorrectly in Real-Address Mode or Virtual-8086 Mode
TGL002	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	Debug Exceptions May be Lost or Misreported When MOV SS or POP SS Instruction is Not Followed by a Write to SP
TGL003	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	CPUID L2 Cache Information May be Inaccurate
TGL004	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	Placing Posted-Interrupt Descriptors Within the PMR2R May Result in a Processor Hang
TGL005	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	Intel® PT CBR Packet May be Delayed or Dropped
TGL006	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	Intel® PT TTP or FUP Packets May be Dropped Without OVF Packet
TGL007	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	Overflow Flag in IA32_MCO_STATUS MSR May be Incorrectly Set
TGL008	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	An Exception During a 32-bit Mode Task Switch With CET Enabled May Lead to an Incorrect TSS Busy Flag Value
TGL009	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	Exit Qualification For EPT Violations Incorrectly Indicate On Instruction Fetches That the Guest-Physical Address Was Writable
TGL010	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	Processor May Generate Spurious Page Faults On Shadow Stack Pages
TGL011	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	HDMI 1.4 Inter-Pair Skew Test May Fail
TGL012	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	Intel® PT ToPA Tables Read From Non-Cacheable Memory During an Intel® TSX Transaction May Lead to Processor Hang
TGL013	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	Performing an XACQUIRE to an Intel® PT ToPA Table May Lead to Processor Hang
TGL014	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	PECI Frequency Limited to 3.2Kbps-1Mbps
TGL015	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	No Fix	PCIe Gen4 JTOL – Jitter Tolerance Compliance Test May Fail
TGL016	Fixed	Fixed	N/A	N/A	N/A	N/A	N/A	Custom Moxi Exit To Exit When Deact or CS

TGL063	Branch Predictor May Produce Incorrect Instruction Pointer
Problem	Under complex microarchitectural conditions, the branch predictor may produce an incorrect instruction pointer leading to unpredictable system behavior.
Implication	Due to this erratum, the system may exhibit unpredictable behavior.
Workaround	It may be possible for BIOS to contain a workaround for this erratum.
Status	For the steppings affected, refer to the Summary Table of Changes .

TGL064	Processor May Encrypt TME Exclude Range if Mapped to Remap Range
Problem	The processor accesses to TME exclude range may be encrypted but not decrypted if mapped to remap range.
Implication	Due to this erratum, the processor exclude range it may be encrypted but may but not decrypted if mapped to remap range.
Workaround	It may be possible for BIOS to workaround this erratum.
Status	For the steppings affected, refer to the Summary Table of Changes .

TGL065	xHCI Force Header Command Incorrect Return Code
Problem	The xHCI controller does not return the correct completion code for the Force Header Command as defined in the Section 4.6.16 of the eXtensible Host Controller Interface for Universal Serial Bus (xHCI) Requirements Specification Rev 1.2.
Implication	xHCI CV TD4.12 - Force Header Command Test may report an error. Intel® has obtained a waiver for TD 4.12. The Force Header Command is only used by the USB-IF Command Verifier (xHCI CV) tool for device testing. There are no known functional failures due to this erratum.
Workaround	None identified.
Status	For the steppings affected, refer to the Summary Table of Changes .

TGL066	USB Type-C Monitor Removal May Result In System Hang
--------	--



https://edc.intel.com/content/www/us/en/secure/design/confidential/products-and-solutions/processors-and-chipsets/tiger-lake/11th-generation-intel-core-processor-family-specification-update/summary-tables-of-changes/#_Summary_Tables_of より

Windows Update でこっそり更新されていたりもする

Intel 製マイクロコードの更新プログラムの概要

適用対象: Windows Server 2019, all versions, Windows 10, version 1809, Windows 10, version 1803, [詳細](#)

Intel 製マイクロコードの更新プログラム

マイクロソフトは、スペクター パリアント 2 (CVE 2017-5715 ["ブランチ ターゲット インジェクション"]) に関連する Intel による検証済みのマイクロコードの更新プログラムをリリースしています。

次の表は、Windows バージョン別のサポート技術情報一覧です。サポート技術情報には、リリースされている Intel 製マイクロコードの更新プログラムが CPU 別に記載されています。

サポート技術情報番号と説明	Windows のバージョン	Source
KB4100347 Intel 製マイクロコードの更新プログラム	Windows 10 Version 1803、Windows Server Version 1803	Windows Update、Windows Server Update Services、Microsoft Update カタログ
KB4090007 Intel 製マイクロコードの更新プログラム	Windows 10 Version 1709 および Windows Server 2016 Version 1709	Windows Update、Windows Server Update Services、Microsoft Update カタログ
KB4091663 Intel 製マイクロコードの更新プログラム	Windows 10, version 1703	Windows Update、Windows Server Update Services、Microsoft Update カタログ

- <https://support.microsoft.com/ja-jp/help/4093836/summary-of-intel-microcode-updates> より

余談：命令の歴史

- RISC-V や MIPS などでは、パイプライン実行を最初から想定
 - 小さいハードで構造ハザードが起きにくいよう設計されている
 - RISC (Reduced Instruction Set Computer) と呼ばれる
 - RISC-V は、「5代目の RISC」という名前
- x86 は、登場時はパイプライン化を考えていなかった
 - 当時は、小数の命令でたくさんのが正義
 - しかし、そのままではパイプライン化は困難
 - CISC (Complex Instruction Set Computer) と呼ばれる
- ARM の R は RISC の R なのだが、ARM は結構 CISC ぽい
 - ARM : Advanced RISC Machine
 - 構造ハザードを凄い勢いで起こす命令が多い

- マイクロ命令への分解により, x86 や ARM はこの問題を（一応）克服
 - 回路規模やエネルギーにおける代償は大きい
 - 互換性が維持できるので, 商業上重要
- x86 や ARM は, 64bit バージョンを作る際に命令の内容をかなり整理した
 - パイプラインが作りやすくなっている
 - 富岳の A64FX では ARM 32bit を切り捨てており（多分）, 大分楽になっているはず

構造ハザードのまとめ

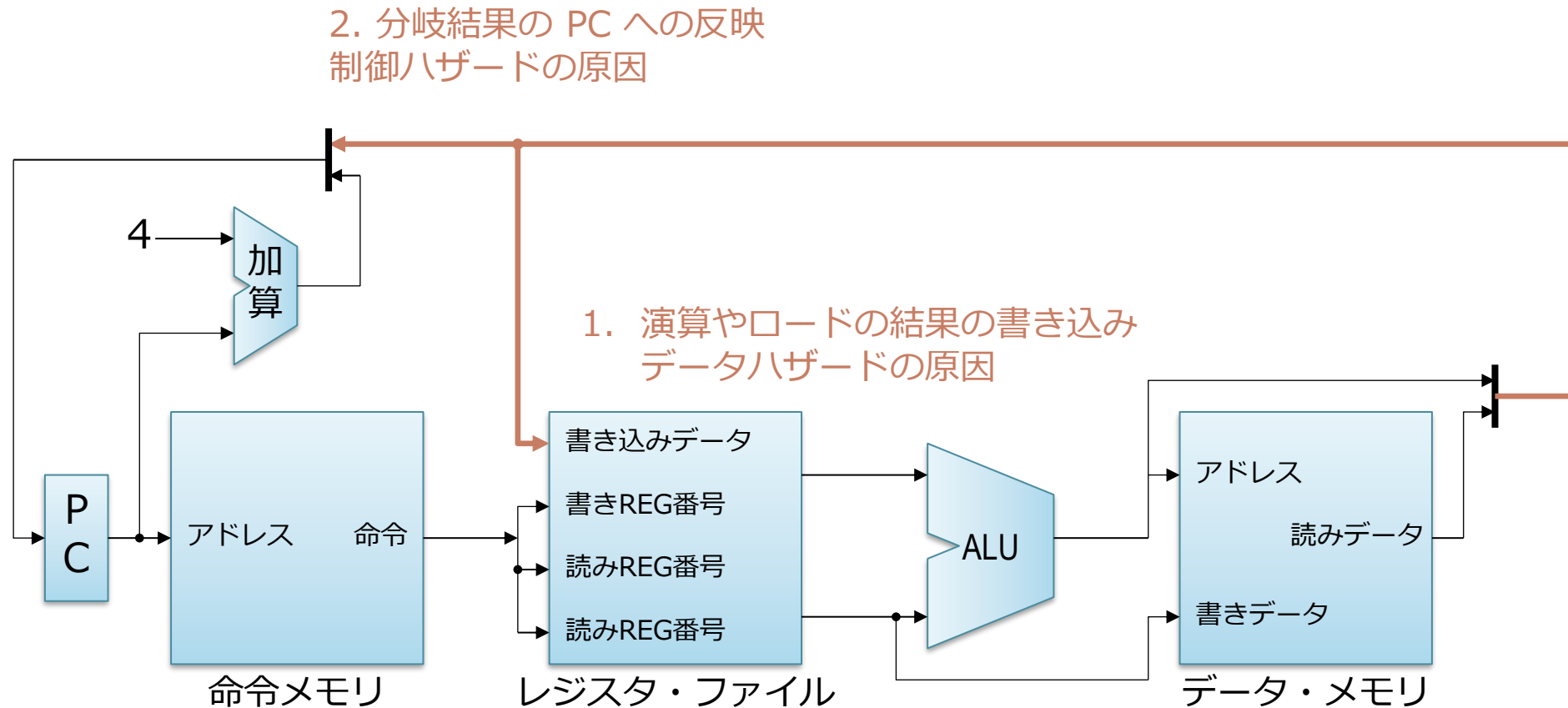
- 構造ハザード：ハード資源の不足に起因
- 解決方法
 1. ハードウェアの増強
 2. 時分割処理
 3. マイクロ命令への変換
- パイプライン・ストール

非構造ハザード

非構造ハザード

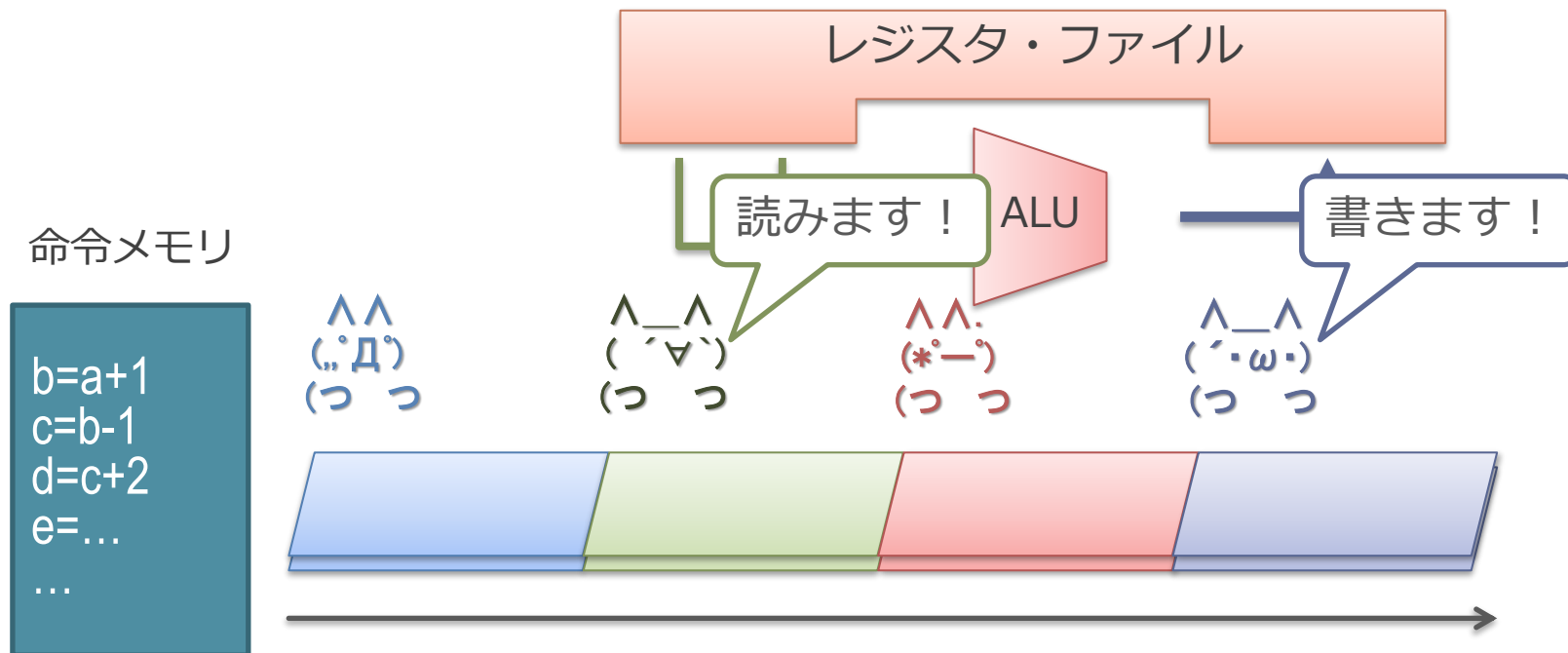
1. 構造ハザード：ハード資源の不足に起因
2. **非構造ハザード：バックエッジに由来**
 - a. データ・ハザード
 - b. 制御ハザード

バックエッジとは：逆方向（右から左）にいく信号



- バックエッジがあるため、命令を単純に流せない場合がある
 - 工場のラインのように、一方向に流せない

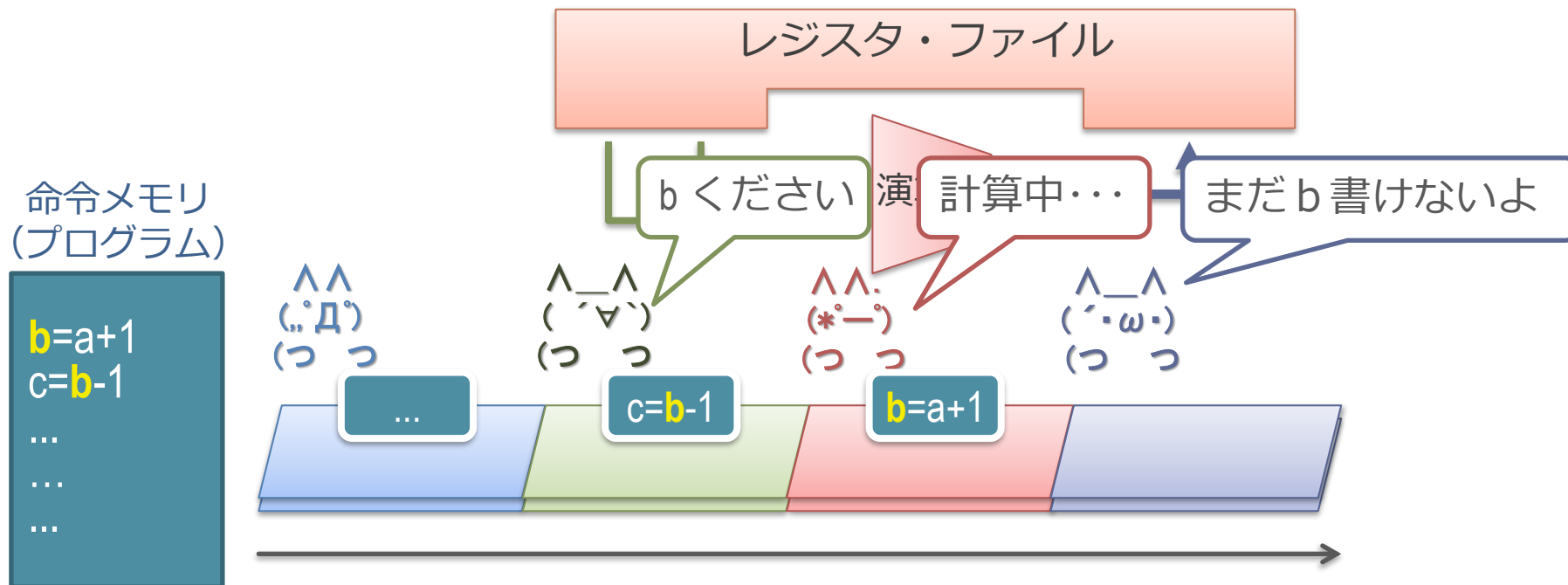
データ・ハザード



■ レジスタ・ファイルへのアクセス

- 演算の入力は(^v)の人がレジスタ・ファイルから読み出す
- 演算の結果は(^ω)の人がレジスタ・ファイルに書き込む

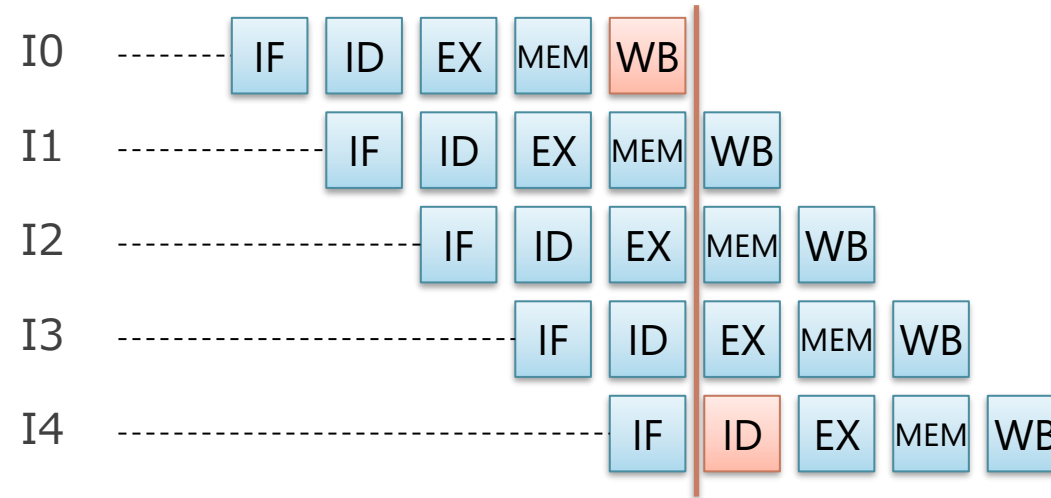
データ・ハザード



■ 直前の命令の結果を使う命令が現れた場合：

- ('▽`) の人が $b=a+1$ の結果を読もうとしても,
- (*°ー) の人がまだ計算中でレジスタ・ファイルに b が書けていない
- (`・ω・) の人が計算結果をかけるのはさらに次のサイクル

データ・ハザード



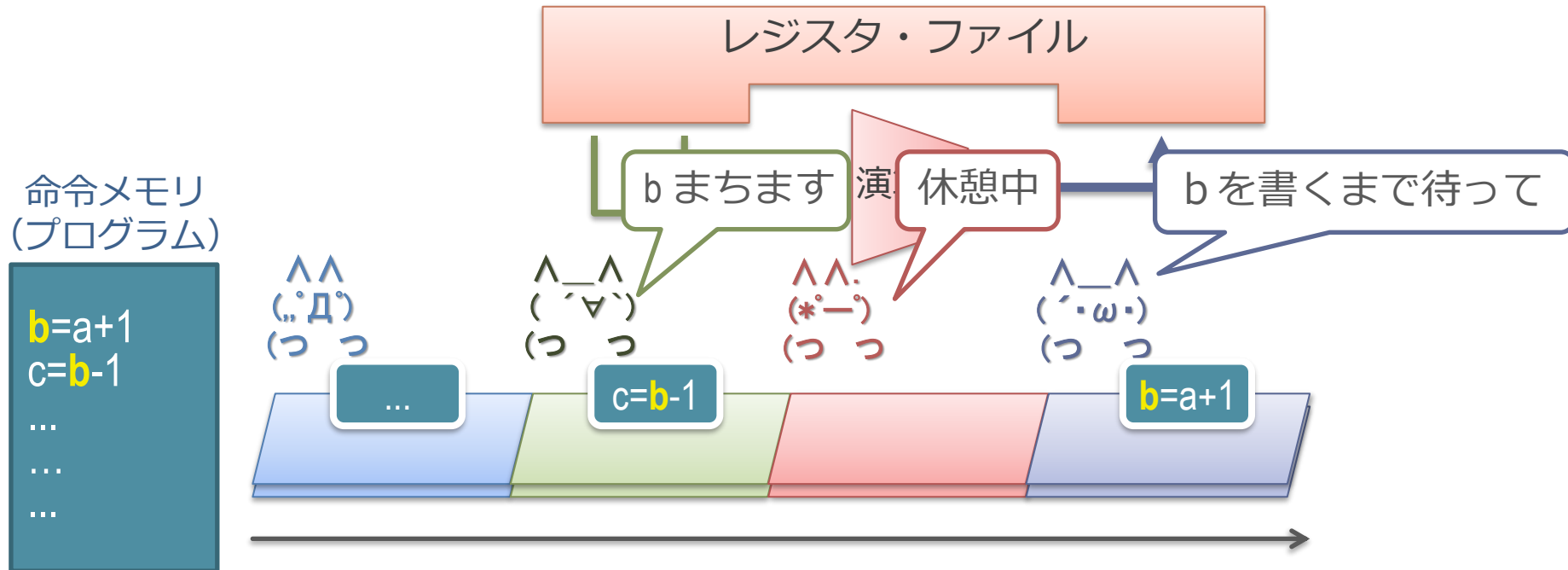
- I0 の WB が終わるまで、その結果はレジスタに書き込まれない
 - I4 までは、その値がレジスタから得られない
 - ID ステージでレジスタを読むため

データ・ハザードの解消方法

■ 解消方法

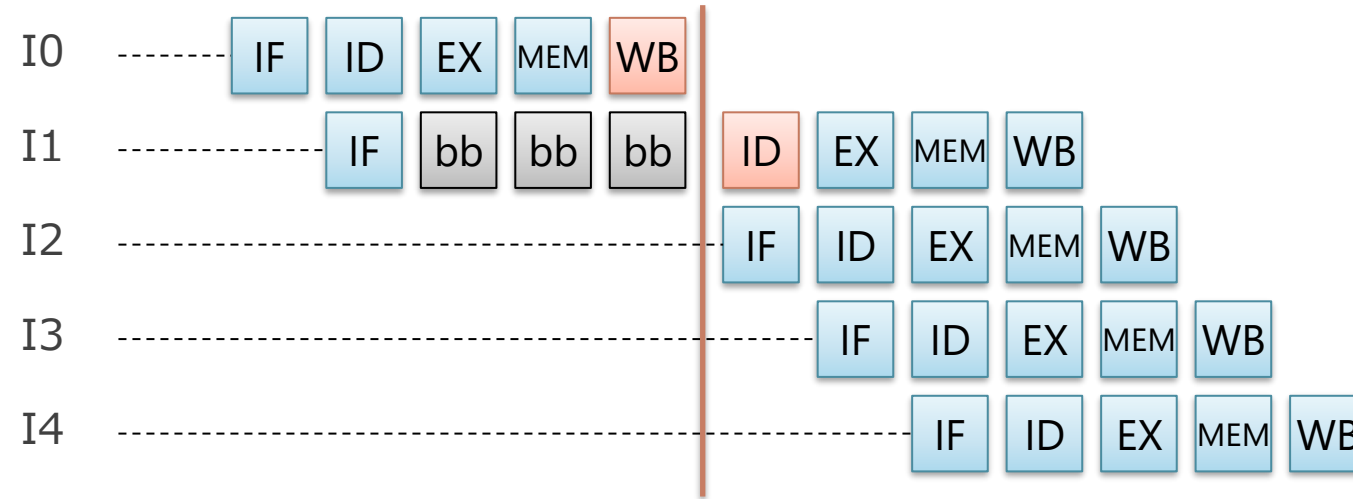
1. ストールさせる
2. 遅延スロット（なにもしない）
3. フォワーディング
4. マルチスレッディング

1. ストール



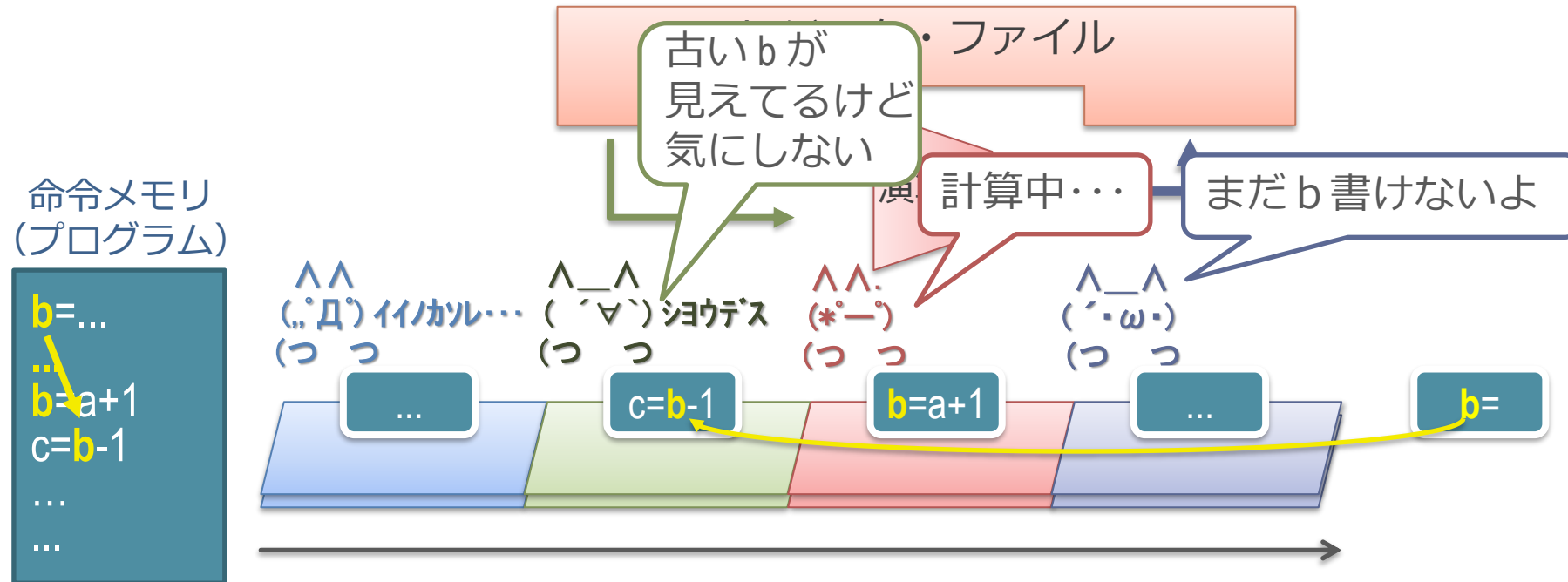
- 直前の命令の結果を使う命令が現れた場合：
 - (っ っ) の人が計算結果をレジスタに書くまで, (っ っ) の人と上流のラインを止める
 - 間にバブル (何もしないステージ) が入る

1. ストールさせる



- I0 の WB が終わるまで、後続の命令を遅らせる
 - I1 の ID が, I0 の WB の右にくるまでストール
 - I1 は I0 の結果を使える
- 欠点 : プログラムの実行がとても遅くなる

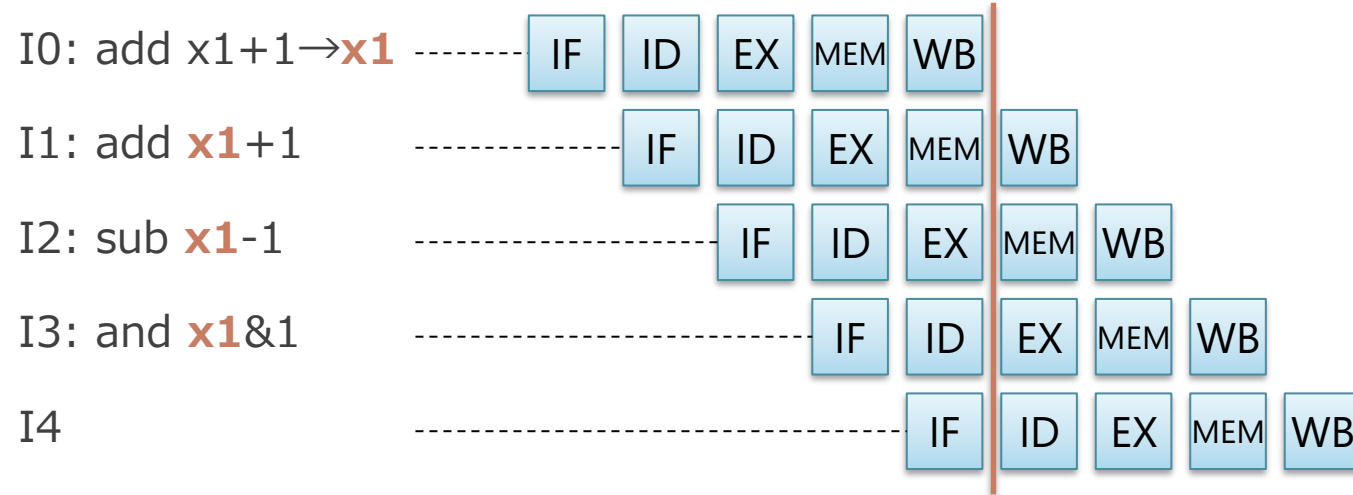
2. 遅延スロット（なにもしない）



■ 直前の命令の結果を使う命令が現れた場合：

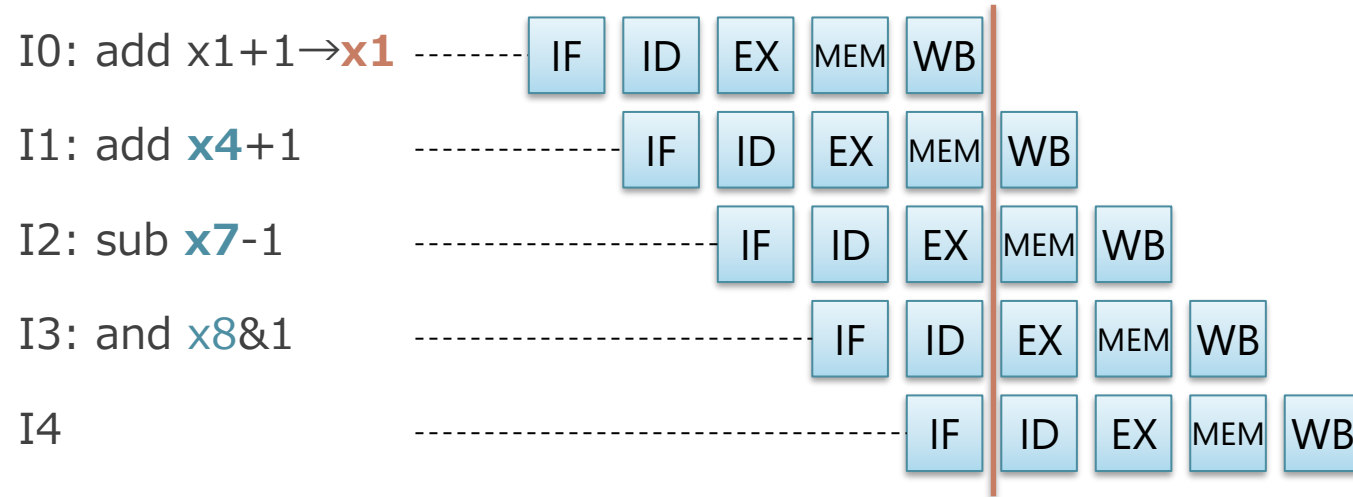
- (' ∇ ') の人は、その時レジスタ・ファイルにある b を気にせず読んでしまう
- 2つ前の命令の結果の b が読めてしまう
 - そう言う仕様ということにする

2. 遅延スロット (=ハードではなにもしない)



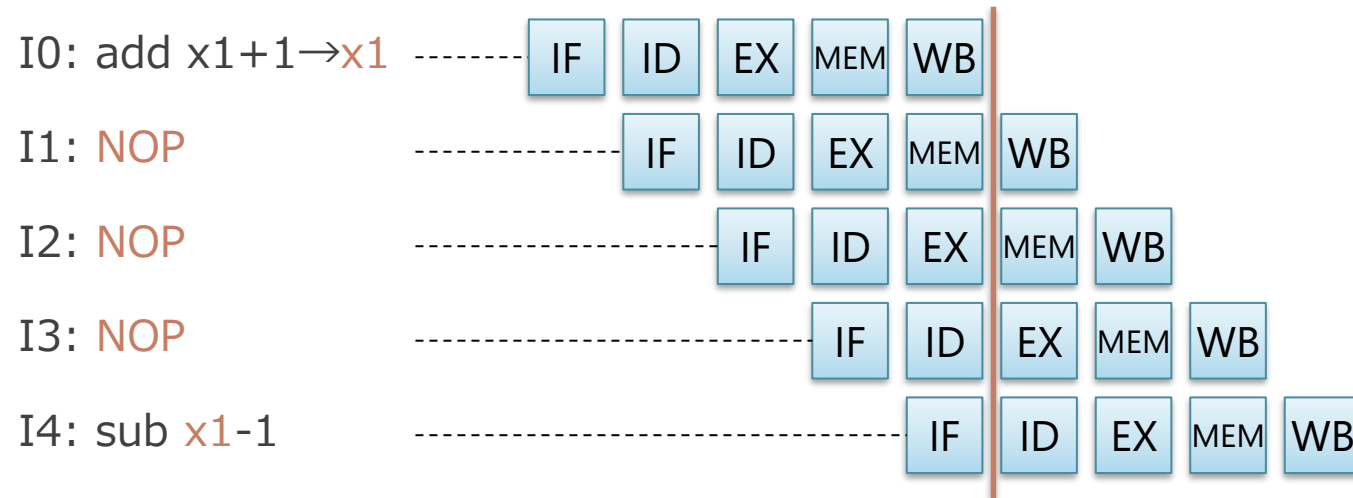
- 特になにも対策せず,
「ある命令の結果は、数命令先まで見えない」という仕様にする
 - 上の例だと I1, I2, I3 は, I0 の結果は見えない
 - I1, I2, I3 には, I0 で add する前の値が見え続ける

2. 遅延スロット (=ハードではなにもしない)



- ここに I0 の結果を使わない命令を入れれば、性能低下はない
 - この部分を「遅延スロット」と呼ぶ
 - この場合、遅延スロットが 3 命令分ある
 - 遅延スロットへの命令挿入はコンパイラががんばる
 - 人力でアセンブリ言語でがんばることもある

NOP の挿入



- もしそのような命令がない場合,
 - NOP (No Operation) と呼ぶ何もしない命令をいれる
 - これもコンパイル時にいれておく必要がある
- 上の例は, x1 に 1 を足した結果を使う以外の処理がなかった場合

遅延スロットの利点

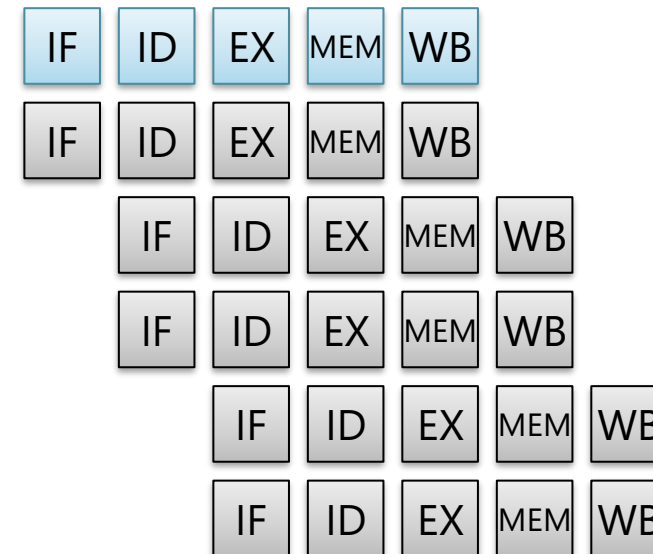
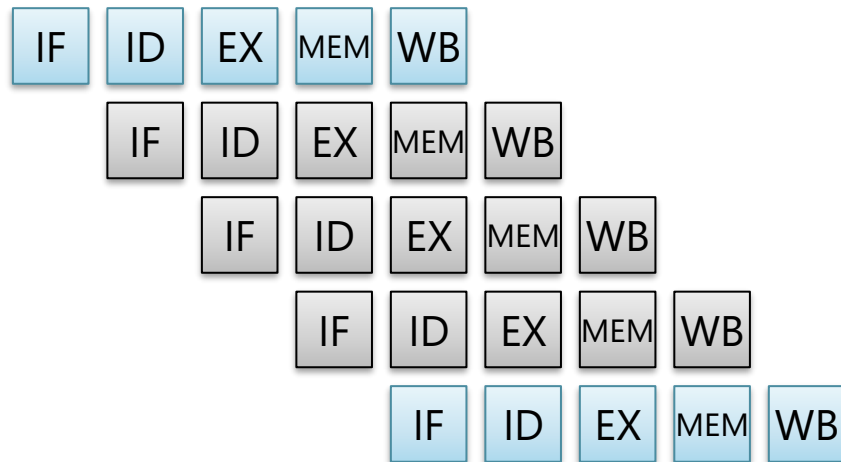
■ 利点：

- なにもしないので，ハードは最も単純
- 並列にできる命令があれば，性能も下がらない

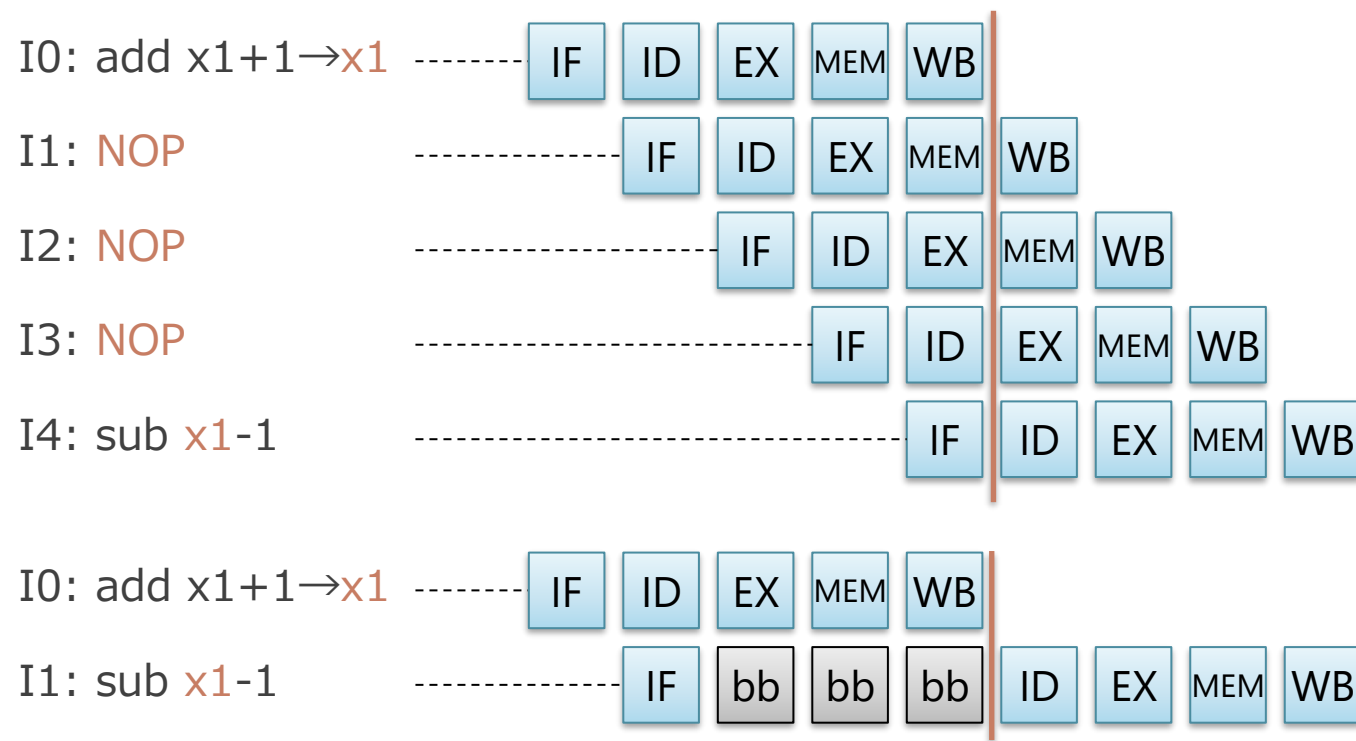
遅延スロットの欠点

- 欠点：「仕様」なので，一度決めると変えられない
 - 後からパイプラインの段数や構造を変えると互換性がなくなる
 - クロックをあげるために，段数を増やせない
 - 複数の命令を同時処理しようとしたときにも互換性がなくなる
 - MIPS では遅延スロットが 1 命令分，仕様として存在
 - 互換性のためにこれを忠実に再現するため後年は逆に複雑化

2 命令同時処理すると，遅延スロットが増える



遅延スロットの欠点2



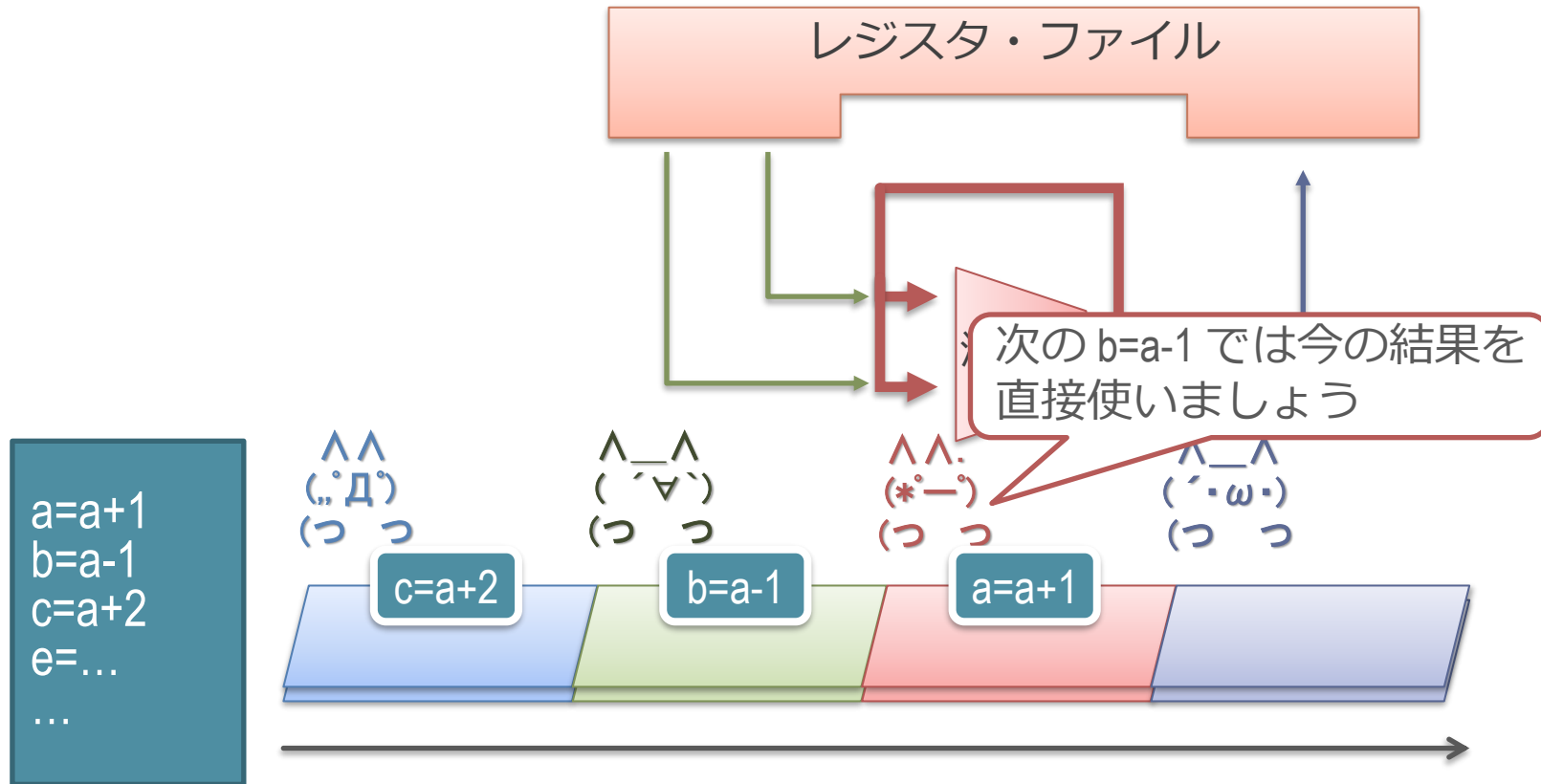
- 欠点2：並列してできる命令が常にあるとは限らない
 - NOPを入れるしかなくなる
 - 実質ストールしてバブルを入れるのと同じになってしまう

データ・ハザードの解消方法

■ 解消方法

1. ストールさせる
2. 遅延スロット（なにもしない）
3. **フォワーディング**
4. マルチスレッディング

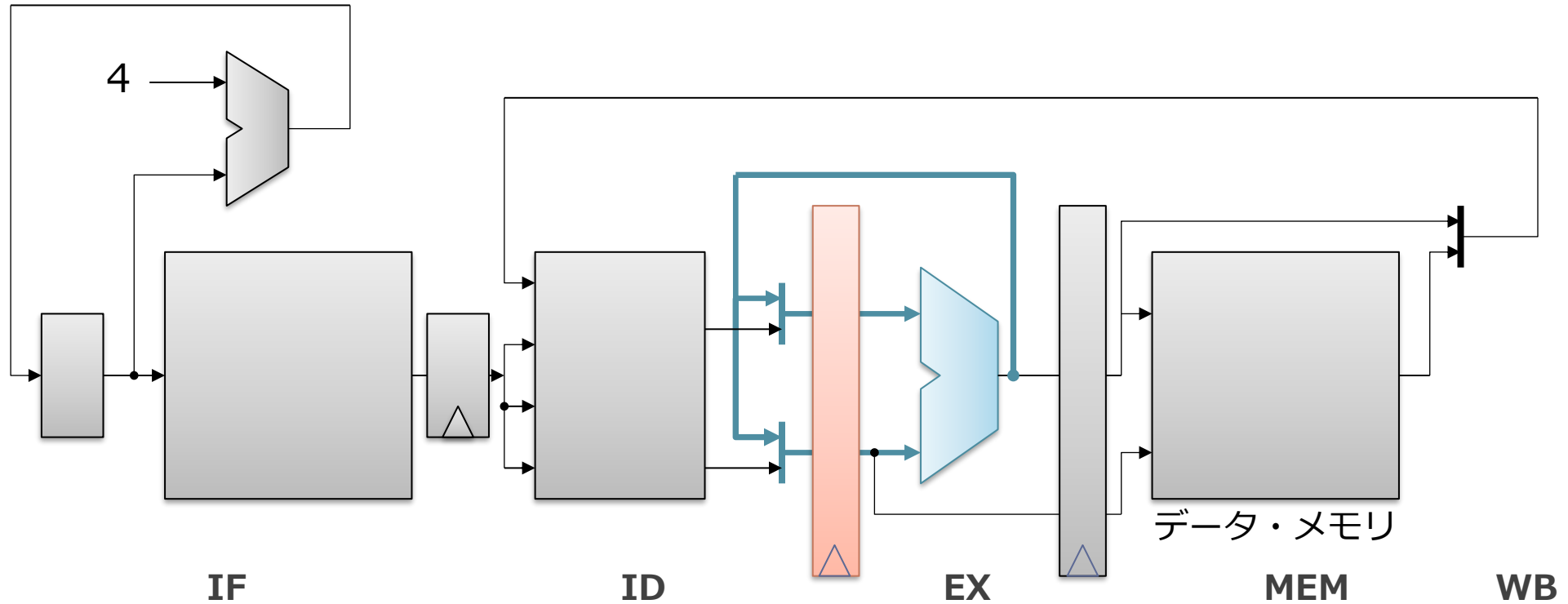
フォワーディング



■ フォワーディング (バイパスとも呼ぶ)

- $(\text{*} \text{ } \text{---})$ の人が、次のサイクルにも結果を使えるようレジスタに書くと同時に手元に結果をおいておく

フォワーディングの回路



- 演算器の結果を, フィードバック
 - レジスタ・ファイルからの読み出し結果と選択して入力に

フォワーディングの利点

■ 利点：

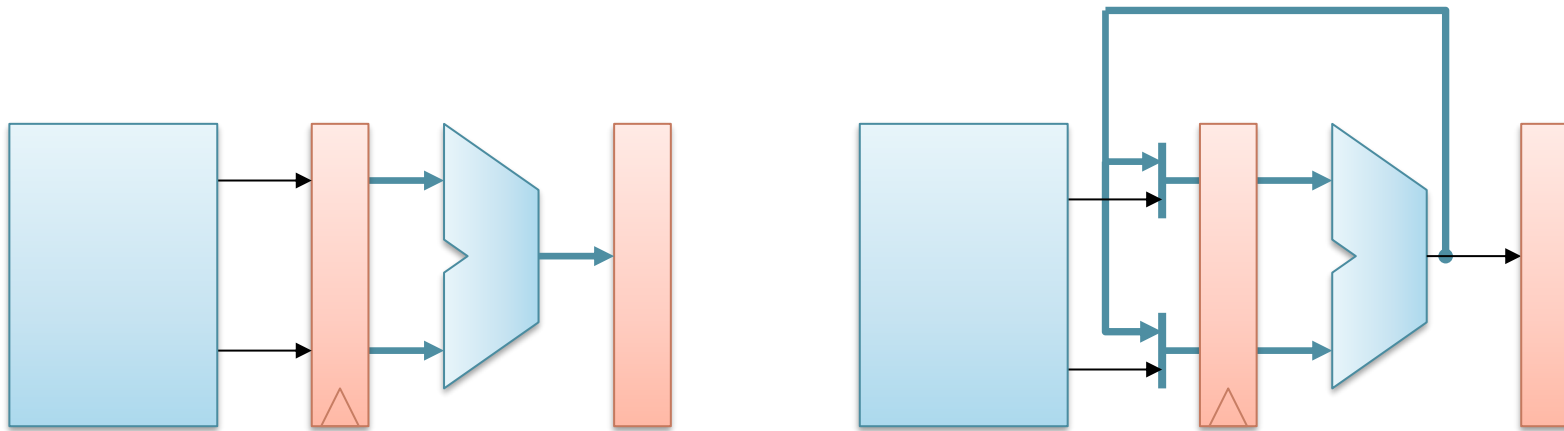
- 依存関係がある命令が連続できてもパイプラインを動かし続けられる
- バブルを発生させることがない

フォワーディングの問題



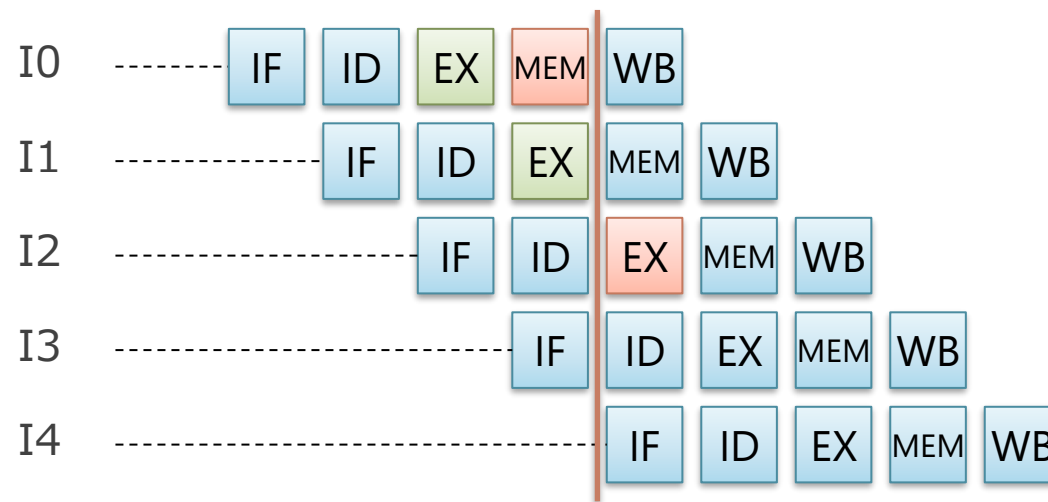
- 演算器とフォワーディングを含むステージは1サイクルで処理する必要がある
 - 実行ステージを演算とフォワーディングに分割すると、バブルを入れてるのと同じになってしまう
 - なので、ここはパイプライン化したくない
- CPU 全体のクリティカル・パスになりやすい
 - 1サイクルに多くの回路を詰め込む必要があるため
 - クロック周波数がここで決まることが多い

フォワーディングの問題



- フォワーディングは、この演算器の部分の遅延を増やしてしまう
 - 太線で描かれた、演算器の入力から始まるループが該当
 - クロック周波数の低下につながる

ロードについては、完全に解決はできない



- ロードではデータ・メモリを読むまでその値は取れない
 - 次の命令は、MEM より後に EX がこないといけない
 - I1 は、I0 のロード結果が見えない
 - この部分はストールや遅延スロットでなんとかすることがおおい

データ・ハザードの解消方法

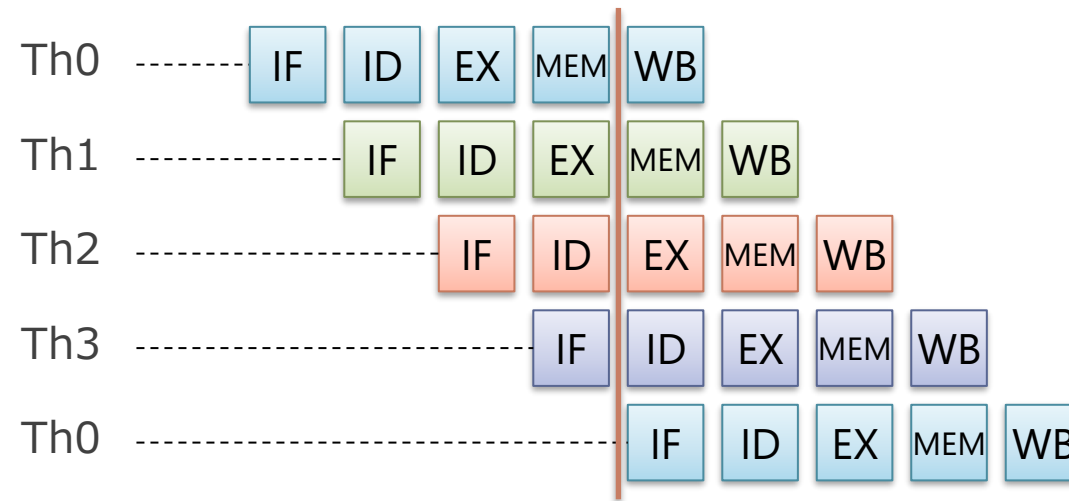
■ 解消方法

1. ストールさせる
2. 遅延スロット（なにもしない）
3. フォワーディング
4. マルチスレッディング

マルチスレッディング (MT: Multi threading)

- 広義のマルチスレッド：
 - コンテキスト (PC やレジスタ) を複数持つこと
- ソフトウェアにおけるマルチスレッド
 - pthread とか
 - 複数のコンテキストが並列して動作
- ハードウェアにおけるマルチスレッド
 - ひとつの CPU 内に複数のコンテキストを複数持つ
 - 次ページの方法は「細粒度マルチスレッディング」と呼ぶ
 - ハードウェアのマルチスレッドは, 他にもいろいろある
 - 「粗粒度 MT」「同時 MT」など

マルチスレッディング



- Th0 から Th3 までの4つのスレッドの命令を順に実行
 - 各スレッドは独立しているので、お互いの結果を読むことはない
- Th0 に戻ってくる頃には、前回の Th0 の結果が書き込まれている

マルチスレッディングの利点と欠点

■ 利点：

- 他の方法のような問題がおきない
 - 理想的にはバブルも発生せず、クロックも落ちない
- 演算器をパイプライン化しても性能に影響がない
 - 他のスレッドを実行して時間をつぶしていればよい

■ 欠点：

1. 動かすスレッドがない場合は、止めておくしかない
 - GPU 等ではスレッドが大量にあるので、問題とならない
 - GPU ではループの各周がスレッドになっているイメージ
2. スレッド数分のレジスタを持つ必要があるのでハードが大きい

データ・ハザードのまとめ

■ 解消方法

1. ストールさせる
2. 遅延スロット（なにもしない）
3. フォワーディング
4. マルチスレッディング

制御ハザード

1. 構造ハザード
2. 非構造ハザード：バックエッジに由来
 - a. データ・ハザード
 - b. 制御ハザード**

分岐命令の処理と制御ハザード



- 「if a > 0」の結果は最終段の(´・ω・)の人まで反映出来ない
 - 先頭は次に a=a+1 と a=a-1 のどちらを取り込めばいいのかわからない

制御ハザードの解消方法

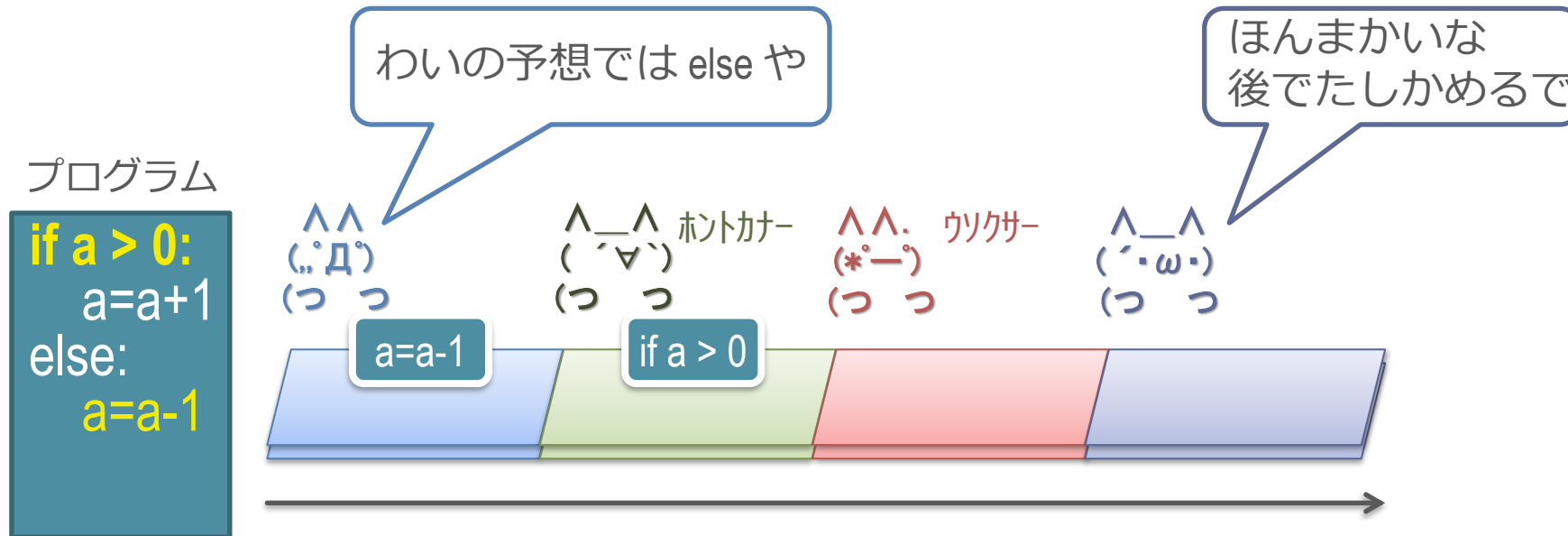
■ 解消方法

1. ストールさせる
2. 遅延スロット（なにもしない）
3. マルチスレッディング

- 上記 3 までは、基本的にデータ・ハザードと同様にして適用できる
 - ただしフォワーディングは制御ハザードでは意味的に無理

4. 分岐予測による投機実行

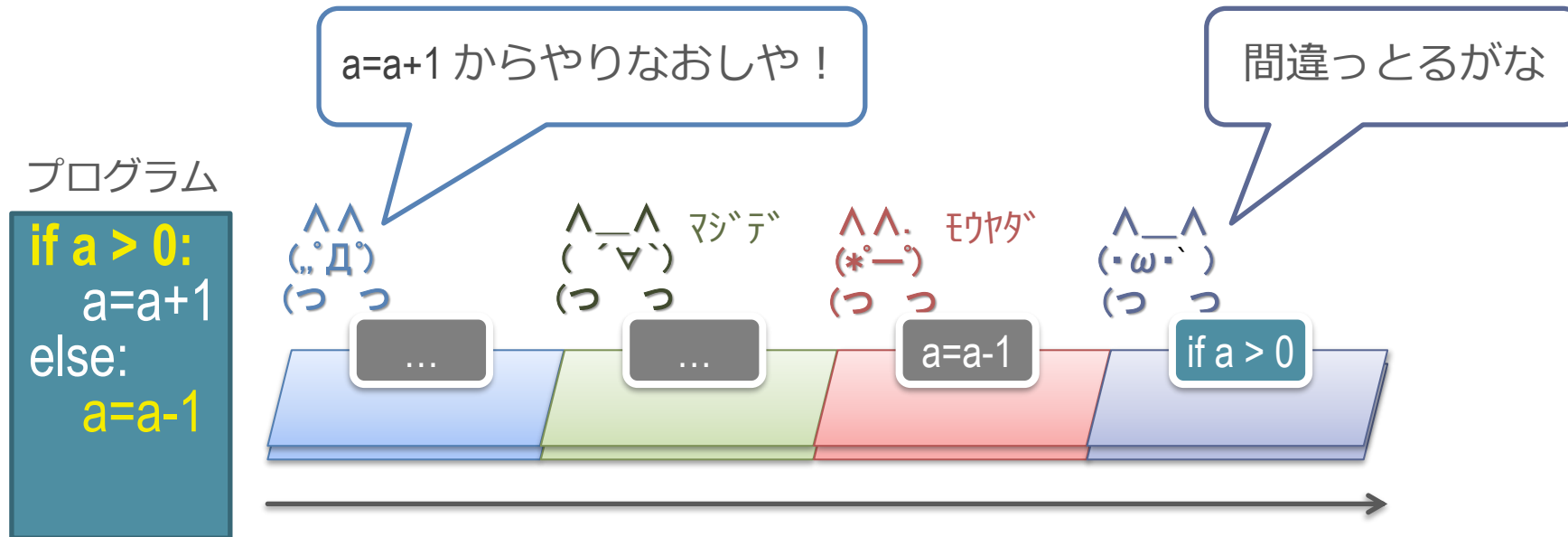
分岐予測



■ 動作

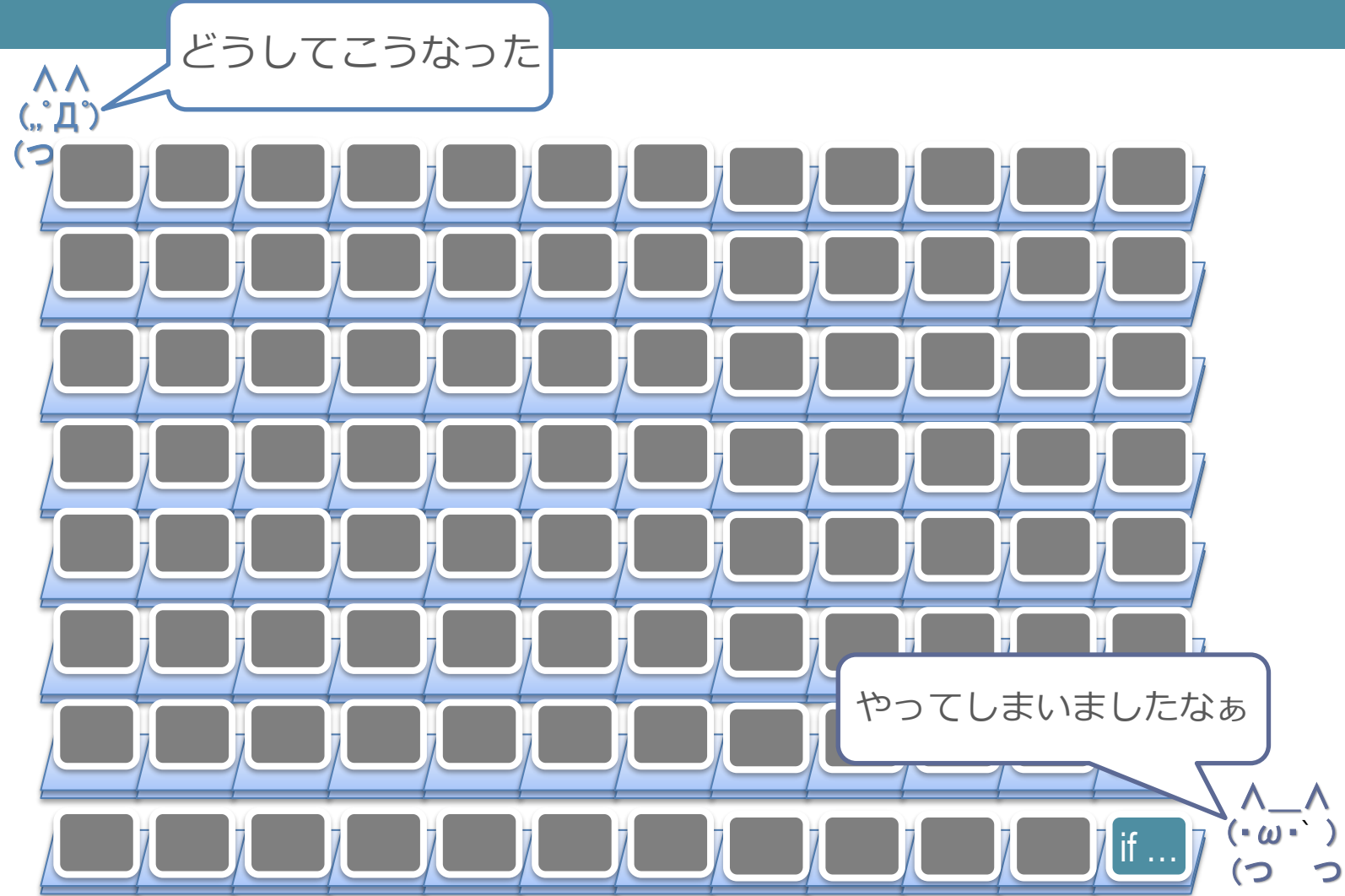
- 「if a > 0」の結果を予測して、命令を取り込む
 - 前はこっちに行ったので、次もこっちに違いないとかで予測
- あとから予測が正しいか確認する

分岐予測ペナルティ



- 予測が間違っていた場合，以降の処理を取り消してやり直す
- この図では，無駄になるのは3命令分

大規模な高性能プロセッサの場合



- 取り消しは最悪数十命令以上に
 - IBM POWER8 だと, 8命令同時 × 10数段

今日のまとめ

- パイプライン
- 各種のハザードと解消方法
 - 構造ハザード
 - 非構造ハザード
 - データ・ハザード
 - 制御ハザード
- 来週は分岐予測の具体的な方法など

出欠と感想

- 本日の講義でよくわかったところ, わからなかったところ, 質問, 感想などを UTOL の出席欄に書いてください
 - LMS の出席を設定するので, そこにお願いします
 - パスワード : arc
- 意見や内容へのリクエストもあったら書いてください
- UTOL の出席の締め切りは来週の講義開始までに設定してあります
 - 仕様上「遅刻」表示になりますが, 特に減点等しません
 - 来週の講義開始までは感想や質問などを受け付けます

- 以下のスプレッドシートの自分の席のところに学籍番号を書いてください
 - https://docs.google.com/spreadsheets/d/1noH3Y2b1tS621wzDHcUIBpJrJEWeqVijQy_RZ25WxNs/edit?usp=sharing
 - 学生側から見た場合の配置で（画面上が教団側），席の対応する行と列のところに書いてください

質問や感想

- 僕は自作PCが趣味ですが、今回の前半でわりと共感できる話があって嬉しいかったです。特に今まではIntel信者でして、自前のPCは12世代のモノをのせています。Intelは第12世代以降、スマートフォンで一般的な「Pコア」と「Eコア」を組み合わせたヘテロジニアス構成を採用しているらしく、今回の話がまさに刺さりそうな感じでした。実際授業のあと調べてみましたが、AMDとの競合を勝つために限界ギリギリまで電圧を上げているらしく、その原因で13世代で大規模な初期不良や、高電力消費に繋がっているらしいです。

- 非同期回路やウェーブパイプラインのところで設計&動作が極めて難しいと書いていますが、こっちはどんなユースケースがあってどんな利点があるのかが気になります

- 学祭などのイベントにおいて、「パイプライン化してスループットを上げる」という発想に至らない人が多くて、「パイプライン化」や「律速」が常識になって欲しいと時折思います。

質問とか回答など

- 主流ではないとはいえ、パイプラインをFFで区切らずに実現することができるのか、と少し衝撃でした
- 確かに理論上はできそうですが、実際に製品レベルで制御できた例があるのか気になりました

- FPGAが演算自体においては効率が悪いというのは意外だった。その反面、CPUにおいては特に命令制御部分が大部分を占めるというのも意外だった。

質問とか回答など

- 出席確認専用ハードウェア開発したい（特にアイデアはないです）
- （正当な理由がなくても）オンラインで受けられるようになりたい、理由ない場合出席点半減とかでもよいので、、、

- ムーアの法則のオーダーをコンデンサのエネルギー，単位面積あたりの個数，周波数から計算して，電圧の閾値の問題から周波数をこれ以上上げにくい理由を解釈するパートが面白かった.
- あとしょぼんとかしいとかが可愛い.

- クロックによる同期化を使わずにパイプラインを作る場合気合いで遅延を合わせると聞いた時、あまりにも力技すぎて、それをやっている研究がすごく気になりました。

- 今回の講義を通して、この授業全体が消費電力という観点を軸に構成されていることに改めて気づきました。消費電力は自分の研究関心とも近いいため、特に興味深く感じました。

- FPUやキャッシュがデカすぎて驚きました。Cerebrasもchipがデカすぎになる理由がわかりました。

- 以前オーバークロックを試したときに感じていた「電圧を上げると消費電力や発熱が一気に増える」という現象を、より理論的に理解できました。特に数式から、電圧が二乗で効くという点が印象に残りました。また、周波数を上げるとスイッチング回数も増えるので、オーバークロック時に電力が大きく増える理由がかなり明確になりました。

- FPGAとCPUの比較が行われていたが、(研究でも開発でもどちらにも言えると思うが)じゃあどういった時はFPGAを使って、どういった時はCPUを使うべきなのかのデファクトスタンダードみたいなものはあるのだろうか？多くの人たちの共通認識としてあるのかが気になった

- ムーアの法則関連で、回路のサイズ以外に回路自体の設計による改善などはないでしょうか。既にチップの設計は収束されているのでしょうか。

- パイプラインの説明は5ステージでされることが多いと思うのですが、実際に使われているCPUなどではどのくらいのステージ数に分割されているのか気になりました。

- FPGAでは専用回路では優秀だが、やることが増えるほど周波数の問題で結局CPUで回したほうがよい(要約)というお話をお聞きして、ASICとかで載せる分にはあまりデメリットがないのかな? という気持ちになりました。 もっともただただCPUと同じレイヤに落ちているだけでデメリットとかで議論する話なのか、という感じがですが.....

- 以前まで、「FPGAは電力効率が悪い」という記述の意味を理解できていなかったが、LUTとしての論理回路実現がトランジスタを贅沢に使用するのが原因だという点が、個人的には明快だった。

- 2000年以前は電圧低下できていたのは何に起因するものなのか気になった。

- aco-shioya-02.pdfの講義資料の52pあたりで、水流と位置エネルギーに例えて解決してくださってとても直感的に若やすかったのですが、入力よりも高い位置にあるPタンクに、入力されたら水が流れているように見えるのですが、位置エネルギー的にあり得るのでしょうか...勢いよく流れるから上まで流れるのですか...解釈が間違っていたらごめんなさい。

質問とか回答など

- power throttling が今かなり問題になっているというのは、自分の経験上も非常にクリティカルな問題だという理解です。周波数をGHz に載せられないレベルとなると、もはやパフォーマンスエンジニアリングが、理論上の性能を引き上げること以上に電力密度を落とす program mapping にすることも必要になりそうだなと、wafer-scale programming をしている身からは感じました。
- （とはいえ、power-throttling は fine-grained で発生するとすると、事実上対策は難しいですが）
- パイプライン処理に関連する話として、GPUのTMAとか Cerebras の PE みたいに、address generation & data load と compute を overlap させて実行するハードウェア機構（およびその専用命令）というのは最新のトレンドとして面白いなと思っています。

- FPGAで専用回路を組むのは効率が悪いとのことでしたが、チップ評価の前にFPGA評価を取るのが有効な場合とそうでない場合があるのでしょうか？

- 天文シミュレーションのN体計算専用のプロセッサ(GRAPE)が昔はあったのに、最近はやられていないことが個人的に気になっていました。特にFPGAとかで昔よりは簡単にできないのかな?とか考えていたのですが、今回の講義でFPが結構面倒であること、それをFPGAに載せるとハードワイヤしたものより遥かに効率が下がってしまうことを理解して納得しました。